

14.1 Diving into an APT investigation

AWS Cloud Incident Investigation

ALERT NAME: Unusual AWS IAM Activity – High Number of API Calls

ALERT ID: UR-AWS-20180820-0038

TIMESTAMP: 2018-08-20T15:15:04Z

SEVERITY: Medium

SOURCE: AWS CloudTrail via SIEM

DETECTED BY: Behavioral Analytics (Baseline Deviation)

ALERT DESCRIPTION:

The SIEM has detected a spike in API activity from an IAM user account. The user initiated multiple AWS service-related actions within a short timeframe, including calls to sensitive services such as IAM and EC2. This burst of activity deviates significantly from the normal behavior profile for IAM users in this account.

DETECTION LOGIC:

Threshold-based anomaly on CloudTrail eventName frequency by `userIdentity.userName`.

14.1.1 IAM users who accessed AWS

14.1.1 Listing the IAM users who accessed an AWS service

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Since the alert is for "Unusual AWS IAM Activity – High Number of API Calls"

I want to figure out the IAM users that accessed an AWS service in Frothly's AWS environment.

I don't care whether that access was successful or unsuccessful – I just want to know anybody who tried to access resources.

`aws:cloudtrail` is your critical sourcetype, as it contains detailed records of all API activity in an AWS environment

First stage the query

`index=client3 sourcetype=aws:cloudtrail`

Then dial it in as I am looking the actual IAM Users

`index=client3 sourcetype=aws:cloudtrail user_type="IAMUser"`

Now pipe it out using stats and sort alphabetically

`index=client3 sourcetype=aws:cloudtrail user_type="IAMUser" | stats count by userName | sort userName`

userName	count
bstoll	606
btun	73
splunk_access	4062
web_admin	641

14.1.2 AWS API brute force

14.1.2 Detecting an AWS API brute force

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out what field I should use to determine that AWS API activity has occurred without MFA.

- `additionalEventData.MFAUsed` is tied to AWS Management Console logins – that is, when a user signs in through the web interface
- someone is using access keys to run AWS CLI commands or make API calls from an application, the relevant MFA indicator is `userIdentity.sessionContext.attributes.mfaAuthenticated`

First stage the query

`index=client3 sourcetype=aws:cloudtrail`

The broad search for mfa

`index=client3 sourcetype=aws:cloudtrail *mfa*`

Looking into the expanded logs under the nested layout for `userIdentity`, `sessionContext`, `Attributes` and field `mfaAuthenticated`

Not looking for console logins so exclude `eventName="ConsoleLogin"`

`index=client3 sourcetype=aws:cloudtrail NOT eventName="ConsoleLogin"`

Since `mfaAuthenticated` is a nest field to search directly for it I must add in the full JSON path to find the `*mfaAuthenticated=false*`

`index=client3 sourcetype=aws:cloudtrail userIdentity.sessionContext.attributes.mfaAuthenticated="false"`
`NOT eventName="ConsoleLogin"`

`index=client3 sourcetype=aws:cloudtrail userIdentity.sessionContext.attributes.mfaAuthenticated="false"`
`NOT eventName="ConsoleLogin"`

14.1.3 Processor number of web server

14.1.3 Locating the processor number used on the web server

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the processor number used on the web servers.

The goal is to discover the type of processor used and the machine or server using it.

First stage the query

`index=client3 | stats count by sourcetype` to give a broad view of the source types and zero in on hardware/cpu

The zero in on the hardware:

`index=client3 *intel* sourcetype=hardware`

I find `Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz`

Then confirm by

`index=client3 *intel* sourcetype=osquery:results`

`columns: { [-]`

`cpu_brand: Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz`

`E5-2676` is the processor number

The host running this processor is `"gacrux.i-0cc93bade2b3cba63"`

server: `Apache/2.2.34 (Amazon)` IP=`172.16.0.145`

14.1.4 Bud's S3 bucket

14.1.4 Identifying the publicly accessible S3 bucket created by Bud

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Bud has accidentally made an S3 bucket publicly accessible.

There is a particular Event ID associated with the API call that enables that public access.

I want to figure out what that associated Event ID is.

(How the public accessed the Bucket)

Set the query:

index=client3 sourcetype="aws:cloudtrail"

Im looking into S3.. need to investigate ACL events (eventName) via PutBucketAcl and AllUsers

index=client3 sourcetype=aws:cloudtrail eventName="PutBucketAcl" AllUsers

This confirms that the event happened

2018/08/20 13:01:46.000 :

Grantee: { [-]

URI: http://acs.amazonaws.com/groups/global/AllUsers

xmlns:xsi: http://www.w3.org/2001/XMLSchema-instance

xsi:type: Group

}

Permission: READ

}

{ [-]

Grantee: { [-]

URI: http://acs.amazonaws.com/groups/global/AllUsers

xmlns:xsi: http://www.w3.org/2001/XMLSchema-instance

xsi:type: Group

}

Permission: WRITE

}

Then to extract all the events information:

index=client3 sourcetype="aws:cloudtrail" eventName="PutBucketAcl" AllUsers

| table _time eventID requestParameters.bucketName userIdentity.principalId

_time	eventID	requestParameters.bucketName	userIdentity.principalId
2018-08-20 13:01:46	ab45689d-69cd-41e7-8705-5350402cf7ac	frothlywebcode	AIDAJUFKXZ44LV4 EN4MGK

eventID- ab45689d-69cd-41e7-8705-5350402cf7ac

Bucket was public between:

2018-08-20 13:01:46 and 2018-08-20 13:57:54 a total of 56min and 8 seconds or 3368 seconds

14.1.5 Public S3 bucket name

14.1.5 Finding the name of the publicly accessible S3 bucket

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the name of the S3 bucket that Bud accidentally made public.

I want to filter for just S3 events `eventSource="s3.amazonaws.com"`

(What Bucket was Accessed)

Set the query:

`index=client3 sourcetype="aws:cloudtrail"`

Then focus into the detail of the ACL and S3

`index=client3 sourcetype="aws:cloudtrail" eventName="PutBucketAcl" eventSource="s3.amazonaws.com"`

The focus only on AllUsers since that is the vulnerability the query

`index=client3 sourcetype="aws:cloudtrail" eventName="PutBucketAcl" eventSource="s3.amazonaws.com"`

AllUsers

Looking in to the JSON nested fields I look into the request parameters and find the bucketName:

`frothlywebcode`

And for verification I query deeper with a table

`index=client3 sourcetype="aws:cloudtrail" eventName="PutBucketAcl" eventSource="s3.amazonaws.com"`

AllUsers | table _time requestParameters.bucketName userIdentity.userName eventName

_time	requestParameters.bucketName	userIdentity.userName	eventName
2018-08-20 13:01:46	frothlywebcode	bstoll	PutBucketAcl

Table confirms `frothlywebcode` as also identified in section 14.1.4

14.1.6 Accessed text file's name

14.1.6 Determining the name of an accessed text file

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which text file was successfully uploaded into that S3 bucket during its public exposure.

I know from exercise 14.1.4 that the bucket was public between:

2018-08-20 13:01:46 and 2018-08-20 13:57:54 a total of 56min and 8 seconds or 3368 seconds

That gives almost an hour for a TA to take advantage

(What happened to the bucket while public)

This task revolves around AWS S3 bucket monitoring using two crucial log types: `aws:cloudtrail` (the control plane, who changed what?) and `aws:s3:accesslogs` (the data plane, who accessed or modified what content).

I know the bucket `frothlywebcode` the time 2018-08-20 13:01:46 and 2018-08-20 13:57:54 now need the `Aws:s3:accesslogs` and filter for `REST.PUT.OBJECT` to find what was uploaded.

`index=client3 sourcetype="aws:s3:accesslogs" frothlywebcode` date and time filtered as above. 110 events
Then add `operation="REST.PUT.OBJECT"`, down to 4 logs and add `"*.txt"` one result.

Key- `OPEN_BUCKET_PLEASE_FIX.txt`

`uri_path -/OPEN_BUCKET_PLEASE_FIX.txt`

`OPEN_BUCKET_PLEASE_FIX.txt` was uploaded to `frothlywebcode` at 2018-08-20 13:02:44. Only 58 seconds after S3 was made public

14.1.7 Archive file's size

14.1.7 Measuring the size of the uploaded archive file

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out how large (in megabytes) was the .tar.gz file that was uploaded to the S3 bucket while it was publicly accessible.

I already have my time window dialed in and the search query for the sourcetype, bucket and operation:

```
index=client3 sourcetype="aws:s3:accesslogs" frothlywebcode operation="REST.PUT.OBJECT"
```

Now add the `*.tar.gz*` file in question gives us 1 event to review

```
13:04:17.000 2018/08/20: frothly_html_memcached.tar.gz object_size-3076532=\1024\1024= 2.93
```

But to make it even easier I eval it out to a table:

```
| eval size_mb = round(object_size / 1024 / 1024, 2) | table key size_mb
```

```
frothly_html_memcached.tar.gz = 2.93
```

14.2 Coin mining on endpoints

Find Coin Mining on Endpoints

14.2.1 Cryptomining process

14.2.1 Detecting the cryptocurrency mining process

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

An endpoint in Frothly's environment is showing evidence of coin mining activity (!).

I want to figure out the name of the first process to hit 100% CPU usage due to coin mining on a Frothly endpoint running Windows, sourcetype: `perfmonmk:process`.

Set the query:

```
index=client3 sourcetype="perfmonmk:process"
```

Then dial it in to host and pipe out a list of host names I know it a windows host so the suffix -L is needed

```
index="client3" sourcetype="PerfmonMk:Process" host="*-L"
```

```
| stats count by host
```

Only 1 host is showing after query= `BSTOLL-L`

Now I know the host I dig in and look at cpu usage:

```
index="client3" sourcetype="PerfmonMk:Process" host="BSTOLL-L" process_cpu_used_percent=100
```

Which give me 134 events

Now look at it in table format and see what process are involved:

```
index="client3" sourcetype="PerfmonMk:Process" host="BSTOLL-L" process_cpu_used_percent=100
```

```
| table _time host process_name process_id process_cpu_used_percent Creating_Process_ID
```

Since I know I need to look at browsers pivot to look only at browser activity:

```
index="client3" sourcetype="PerfmonMk:Process" host="BSTOLL-L" process_cpu_used_percent=100
```

```
(process_name="*chrome*" OR process_name="*iexplorer*" OR process_name="*firefox*" OR
```

```
process_name="*brave*" OR process_name="*opera*" OR process_name="*edge*")
```

```
| table _time host process_name process_id process_cpu_used_percent Creating_Process_ID
```

And sort by time to get the earliest

Can add `| sort _time` or toggle the time on the table

_time	host	process_name	process_id	process_cpu_used_percent	Creating_Process_ID
2018-08-20 09:36:26	BSTOLL-L	MicrosoftEdgeCP#2	1620	100	792

This was the first time a browser hit 100% CPU but that doesn't mean it is due to mining. Open the event and see what was happening before and after the event. Set time range in the event +/- 5 mins the change to dns stream with this query `index="client3" host="BSTOLL-L" sourcetype="stream:dns" | stats count by query`

Added in a lookup table : `coinminingdomains.csv`.

`index="client3" host="BSTOLL-L" sourcetype="stream:dns"`

`| lookup coinminingdomains.csv Domain AS query OUTPUT Domain AS matched_domain`

`| search matched_domain=*`

Queried no results so this event was not involved in mining. (knew that already by looking at the query :

`index="client3" sourcetype="PerfmonMk:Process" host="BSTOLL-L" process_cpu_used_percent=100`

`(process_name="*chrome*" OR process_name="*iexplorer*" OR process_name="*firefox*" OR process_name="*brave*" OR process_name="*opera*" OR process_name="*edge*")`

`| table _time host process_name process_id process_cpu_used_percent Creating_Process_ID`, earlier and

this was the only Microsoft edge event. The remaining ones are chrome and there are 132 of them all within 10 seconds apart and begin about 4 hours after this one. So time to re query and check the first chrome event in the pattern to see if it was involved using the same steps above.

First

`index="client3" sourcetype="PerfmonMk:Process" host="BSTOLL-L" process_cpu_used_percent=100`

`(process_name="*chrome*" OR process_name="*iexplorer*" OR process_name="*firefox*" OR process_name="*brave*" OR process_name="*opera*" OR process_name="*edge*")`

`| table _time host process_name process_id process_cpu_used_percent Creating_Process_ID | sort _time`

2018-08-20 13:37:50	BSTOLL-L	chrome#5	3400	100	9752
---------------------	----------	----------	------	-----	------

`index="client3" host="BSTOLL-L" sourcetype="stream:dns"`

`| lookup coinminingdomains.csv Domain AS query OUTPUT Domain AS matched_domain`

`| search matched_domain=*`

Upon verifying via

`index="client3" host="BSTOLL-L"`

`(`

`sourcetype="PerfMonMk:Process" process_cpu_used_percent=100`

`(process_name=*chrome* OR process_name=*firefox* OR process_name=*iexplorer* OR process_name=*brave* OR process_name=*opera* OR process_name=*edge*)`

`)`

`OR sourcetype="stream:dns"`

`| search process_cpu_used_percent=100 OR *coin*`

`| table _time sourcetype host instance process_id name src_ip dest_ip`

2018-08-20 13:37:50 BSTOLL-L chrome#5 3400 100 9752 is the first instance of mining.

14.2.2 Cloud init script's packages

14.2.2 Counting packages installed by a cloud initialization script

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Automated configuration tasks are performed after a Frothly EC2 web server instance is launched via autoscaling.

I want to figure out how many packages and dependencies are installed by the EC2 instance's cloud init script when a Frothly web server is auto-scaled. This will be looking into `cloud-init-output` sourcetype and utilizing `rex` to carve out the meaningful data

Setting up the broad query:

`index="client3" sourcetype="cloud-init-output"` gives 23 events to dig into

Since I want to know the packages installed add the filter:

`packages "dependent packages" installed`

Then extract data using `rex` to make it easier to view

`| rex field=_raw`

`"Install\s+(?<installed_packages>\d+)\s+Packages\s+\s+(?<dependent_packages>\d+)\s+Dependent packages\s+"`

`| table _time installed_packages dependent_packages`

_time	installed_packages	dependent_packages
2018-08-20 13:33:24	7	13
2018-08-20 14:23:19	7	13
2018-08-20 14:25:21	7	13

14.2.3 Monero-mining endpoint

14.2.3 Finding the hostname of the endpoint that mined Monero

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the short hostname of the sole Frothly system that successfully mined Monero by focusing on identifying a compromised Windows host that was used to mine Monero cryptocurrency.

First set the query for a windows endpoint "-L" and since we are looking for a host name, a DNS query. `index="client3" sourcetype="stream:dns" host="*-L"` provides 74,973 events

Utilize the lookup of coinminingdomains.csv

`index="client3" host="BSTOLL-L" sourcetype="stream:dns"`

`| lookup coinminingdomains.csv Domain AS query OUTPUT Domain AS matched_domain`

`| search matched_domain=* | table _time sourcetype source host query matched_domain`

This gives me 14 events all involving host BSTOLL-L.

_time	sourcetype	source	host	query	matched_domain
2018-08-20 13:39:20.1 85	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	coinhive.co m	coinhive.co m
2018-08-20 13:39:20.1 85	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	ws019.coinh ive.com	ws019.coin hive.com
2018-08-20 13:39:20.1 85	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	coinhive.co m	coinhive.co m

2018-08-20 13:39:20.1 85	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	ws019.coinh ive.com	ws019.coin hive.com
--------------------------------	------------	--	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	coinhive.co m	coinhive.co m
--------------------------------	------------	--------------------------------	----------	------------------	------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	ws001.coinh ive.com	ws001.coin hive.com
--------------------------------	------------	--------------------------------	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	ws005.coinh ive.com	ws005.coin hive.com
--------------------------------	------------	--------------------------------	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	ws011.coinh ive.com	ws011.coin hive.com
--------------------------------	------------	--------------------------------	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSIntegrity	BSTOLL-L	ws014.coinh ive.com	ws014.coin hive.com
--------------------------------	------------	--------------------------------	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	coinhive.co m	coinhive.co m
--------------------------------	------------	--	----------	------------------	------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	ws001.coinh ive.com	ws001.coin hive.com
--------------------------------	------------	--	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	ws005.coinh ive.com	ws005.coin hive.com
--------------------------------	------------	--	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	ws011.coinh ive.com	ws011.coin hive.com
--------------------------------	------------	--	----------	------------------------	------------------------

2018-08-20 13:38:19.3 59	stream:dns	stream:Splunk_ DNSRequestRe sponse	BSTOLL-L	ws014.coinh ive.com	ws014.coin hive.com
--------------------------------	------------	--	----------	------------------------	------------------------

Now using stats to group queries by host and domain

```
index="client3" host="BSTOLL-L" sourcetype="stream:dns"
```

```
| lookup coinminingdomains.csv Domain AS query OUTPUT Domain AS matched_domain
```

```
| search matched_domain=*
```

```
| stats earliest(_time) AS first_seen latest(_time) AS last_seen count AS total_queries BY host,  
matched_domain
```

```
| eval first_seen=strftime(first_seen, "%Y-%m-%d %H:%M:%S"), last_seen=strftime(last_seen, "%Y-%m-%d  
%H:%M:%S")
```

host	matched_domain	first_seen	last_seen	total_queries
BSTOLL-L	coinhive.com	2018-08-20 13:38:19	2018-08-20 13:39:20	4
BSTOLL-L	ws001.coinhive.co m	2018-08-20 13:38:19	2018-08-20 13:38:19	2
BSTOLL-L	ws005.coinhive.co m	2018-08-20 13:38:19	2018-08-20 13:38:19	2

BSTOLL-L	ws011.coinhive.co m	2018-08-20 13:38:19	2018-08-20 13:38:19	2
BSTOLL-L	ws014.coinhive.co m	2018-08-20 13:38:19	2018-08-20 13:38:19	2
BSTOLL-L	ws019.coinhive.co m	2018-08-20 13:39:20	2018-08-20 13:39:20	2

Now i know it BSTOLL-L pivot to see performance data with `sourcetype=PerfmonMk:Process index=client3 sourcetype=PerfmonMk:Process host="BSTOLL-L"` provides 139,828 events

Narrow down by host, process_name and process_id, running time of atleast 5 minutes, how often the process was at a high CPU and commute the risk_score.

```
index=client3 sourcetype=PerfmonMk:Process host="BSTOLL-L"
| eval high_cpu=if(process_cpu_used_percent >= 90, 1, 0)
| stats count, earliest(_time) as et, latest(_time) as lt, max(Elapsed_Time) as elapsed,
min(process_cpu_used_percent) as min_cpu,
max(process_cpu_used_percent) as max_cpu,
avg(process_cpu_used_percent) as avg_cpu,
sum(high_cpu) as high_cpu_samples
by host, process_name, process_id
| where elapsed >= 300
| eval risk_score = (high_cpu_samples / elapsed) * 100
| convert ctime(et), ctime(lt)
| sort - risk_score
```

Again **BSTOLL-L** is the Winner.

14.2.4 Cryptocurrency mining destinations

14.2.4 Calculating the number of cryptocurrency mining destinations

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the total number of cryptocurrency mining destinations accessed by Frothy endpoints. This time, utilizing dc() in stats to get the distinct count.

Stage the query as per usual

```
index="client3" sourcetype="stream:dns"
```

Now use lookup to match the names with coinminingdomains

```
index="client3" sourcetype="stream:dns"
```

```
| lookup coinminingdomains.csv Domain AS name OUTPUT Domain AS matched_domain
```

```
| search matched_domain=*
```

Spit it out into a read easy table

```
| table _time sourcetype source host name matched_domain
```

Now use stats dc() to calculate the # of unique destinations

```
| stats dc(matched_domain) as distinct_mining_destinations by host
```

host	distinct_mining_destinations
BSTOLL-L	6
MKRAEUS-L	6

distinct_mining_destinations
6

Now to display my evidence in a detailed format

```
index="client3" sourcetype="stream:dns"
```

```
| lookup coinminingdomains.csv Domain AS name OUTPUT Domain AS matched_domain
```

```
| search matched_domain=*
```

```
| table _time sourcetype source host name matched_domain
```

```
| stats min(_time) as first_seen, max(_time) as last_seen, count by host, matched_domain
```

```
| convert ctime(first_seen), ctime(last_seen)
```

```
| table host matched_domain count first_seen last_seen
```

host	matched_domain	count	first_seen	last_seen
BSTOLL-L	coinhive.com	4	08/20/2018 13:37:33.910	08/20/2018 13:38:39.701
BSTOLL-L	ws001.coinhive.com	1	08/20/2018 13:37:49.615	08/20/2018 13:37:49.615
BSTOLL-L	ws005.coinhive.com	1	08/20/2018 13:38:06.776	08/20/2018 13:38:06.776
BSTOLL-L	ws011.coinhive.com	1	08/20/2018 13:38:03.958	08/20/2018 13:38:03.958
BSTOLL-L	ws014.coinhive.com	1	08/20/2018 13:37:34.251	08/20/2018 13:37:34.251
BSTOLL-L	ws019.coinhive.com	1	08/20/2018 13:38:38.131	08/20/2018 13:38:38.131
MKRAEUS-L	coinhive.com	4	08/20/2018 13:27:17.402	08/20/2018 13:28:23.193

MKRAEUS-L	ws001.coinhive.com	1	08/20/2018 13:27:33.106	08/20/2018 13:27:33.106
MKRAEUS-L	ws005.coinhive.com	1	08/20/2018 13:27:50.267	08/20/2018 13:27:50.267
MKRAEUS-L	ws011.coinhive.com	1	08/20/2018 13:27:47.45	08/20/2018 13:27:47.45
MKRAEUS-L	ws014.coinhive.com	1	08/20/2018 13:27:17.743	08/20/2018 13:27:17.743
MKRAEUS-L	ws019.coinhive.com	1	08/20/2018 13:28:21.622	08/20/2018 13:28:21.622

14.2.5 Suricata detection ID

14.2.5 Determining the ID of the first Suricata detection

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Using Splunk's event order functions, I want to figure out the earliest recorded signature ID of the coin miner threat in Frothly's Symantec Endpoint Protection (SEP) logs.

For this working with `symantec:ep:security:file`

Query the search first with

```
index="client3" sourcetype="symantec:ep:security:file"
```

Providing 47 events

Filter in the keywords utilizing rex like coinminer, cryptominer, and monero

```
index="client3" sourcetype="symantec:ep:security:file"
```

```
| rex field=_raw "CIDS Signature string:\s*(?<CIDS_Signature>[^\s]+)"
```

```
| where match(CIDS_Signature, "(?i)coinminer|cryptominer|monero")
```

```
| table _time dvc user Domain_Name file_name Intrusion_URL CIDS_Signature CIDS_Signature_ID category  
Traffic_Direction action dest_ip src_ip severity vendor_product
```

```
2018-08-20 13:37:40    BTUN-L BillyTun AzureAD    MICROSOFTEDGECP.EXE  
www.brewertalk.com/forumdisplay.php?fid=9    Web Attack: JSCoinminer Download 8    30358    Web  
Attack    Inboundblocked 192.168.3.130    54.67.127.227    high    Symantec Endpoint Protection
```

30358 is the earliest signature ID

14.2.6 Malware severity via OSINT

14.2.6 Using OSINT to look up the severity of the malware

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the severity level assigned to this coin miner threat (as documented on Symantec's official website).

By looking at the event log from 14.2.5, I know the CIDS_Signature is Web Attack: JSCoinminer Download 8. So completing a search from Broadcom, Web Attack: JSCoinminer Download 8. Provides webpage <https://www.broadcom.com/support/security-center/attacksignatures/detail?asid=30358> reviewing this OSINT give me a **Medium** severity

14.2.7 Coinminer-defeating endpoint

14.2.7 Identifying the endpoint that defeated the coin miner

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which Frothy endpoint successfully overcame the cryptocurrency attack. I'm interested in that device's short hostname.

Again I'll be working with `symantec:ep:security:file`

Beginning with

```
index="client3" sourcetype="symantec:*" *coinminer OR mine
```

 giving the same 46 events

Now narrow to the CIDS_Signature_String:

```
index="client3" sourcetype="symantec:ep:security:file" CIDS_Signature_String="*coinminer*
```

Now look to see where the system stopped "blocked" that attempt and identify the host

```
index=client3 sourcetype="symantec:ep:security:file" action="blocked"
```

```
| table _time Host_Name CIDS_Signature_String action
```

The only host listed is **BTUN-L**. Blocking 46 events starting at 2018-08-20 13:37:40 and ending 2018-08-20 13:46:47.

Final verification:

```
index=client3 sourcetype="symantec:ep:security:file" action="blocked" | stats count by Host_Name
```

Confirms **BTUN-L** with 46 attempts

14.2.8 Different OS endpoint

14.2.8 Finding the endpoint with a different OS version

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which endpoint runs a different edition of Windows compared to the rest. Let's identify it by FQDN.

I'll be using `stats` to compare hosts across attributes and parsing the FQDN

Start with a broad search to find OS-related fields:

```
index="client3" sourcetype=* CASE(Windows) (os OR operating) (ver OR version)
```

3,490 events

WinEventLog:Microsoft-Windows-Sysmon/Operational	3,269	93.668%
operatingsystem	204	5.845%
code42://frothly_code42_input	17	0.487%

OS versions given:

Microsoft Windows	3,269	94.126%
Microsoft Windows 10 Pro	174	5.01%
Microsoft Windows 10 Enterprise30		0.864%

Looking into the source types, 2 options available

However, `WinHostMon` breaks down everything I am looking for

```
8/20/18      Type=OperatingSystem
3:17:23.000 PM OS="Microsoft Windows 10 Pro"
              Architecture="64-bit"
              Version="10.0.17134"
              BuildNumber="17134"
```

[Show all 22 lines](#)

```
host = FYODOR-L
index = client3
source = operatingsystem
sourcetype = WinHostMon
```

Now I pivot off this info to find a different OS being utilized:

```
index="client3" sourcetype=WinHostMon  
| stats values(OS) values(Version) by host
```

host	values(OS)	values(Version)
ABUNGST-L	Microsoft Windows 10 Pro	10.0.17134
BGIST-L	Microsoft Windows 10 Pro	10.0.17134
BSTOLL-L	Microsoft Windows 10 Enterprise	10.0.17134
BTUN-L	Microsoft Windows 10 Pro	10.0.17134
FYODOR-L	Microsoft Windows 10 Pro	10.0.17134
JWORTOS-L	Microsoft Windows 10 Pro	10.0.17134
MKRAEUS-L	Microsoft Windows 10 Pro	10.0.17134
PCERF-L	Microsoft Windows 10 Pro	10.0.17134

BSTOLL-L is running a different OS

Now pivot to match up with the FQDN

```
index="client3" sourcetype=WinHostMon  
| stats values(OS) as OS  
values(Version) as Version  
by host  
| join type=left host [  
search index="client3" sourcetype=xmlwineventlog  
source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
```

```
| rex field=_raw "<Computer>(?!<FQDN>[^\<]+)</Computer>"
| stats earliest(_time) as firstSeen values(FQDN) as FQDNs by host
]
| table host OS Version FQDNs
```

ABUNGST-L	Microsoft Windows 10 Pro	10.0.17134	ABUNGST-L.froth.ly
BGIST-L	Microsoft Windows 10 Pro	10.0.17134	BGIST-L.froth.ly
BSTOLL-L	Microsoft Windows 10 Enterprise	10.0.17134	BSTOLL-L.froth.ly
BTUN-L	Microsoft Windows 10 Pro	10.0.17134	BTUN-L.froth.ly
FYODOR-L	Microsoft Windows 10 Pro	10.0.17134	FYODOR-L.froth.ly
JWORTOS-L	Microsoft Windows 10 Pro	10.0.17134	JWORTOS-L.froth.ly
MKRAEUS-L	Microsoft Windows 10 Pro	10.0.17134	MKRAEUS-L.froth.ly
PCERF-L	Microsoft Windows 10 Pro	10.0.17134	PCERF-L.froth.ly

14.2.9 Cryptomining duration via flow logs

14.2.9 Measuring duration of cryptomining from flow logs

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the duration in seconds that the endpoint mined Monero according to Cisco NVM flow logs.

For this the source type is `sourcetype="ciscovmflowdata"` a flow log that summarizes network communications directly.

Combining with known indicator `coinhive` and compromised host `BSTOLL-L`. I will zero in on the events in question.

Staging the query:

```
index="client3" source="ciscovmflowdata"
```

Provides 78,459 events

Adding `coinhive` to the query

```
index="client3" source="ciscovmflowdata" coinhive
```

 gives me 6 events

Now to aggregate the earliest and latest flow times

```
index="client3" source="ciscovmflowdata"
```

```
| search coinhive.com
```

```
| eval fss = tonumber(fss), fes = tonumber(fes)
```

```
| stats
```

```
    min(fss) AS starttime
```

```
    max(fes) AS endtime
```

```
BY host sa da dh
```

```
| table host sa da dh starttime endtime
```

splunkhvf.froth	192.168.247.	104.20.208.	coinhive.com	153477	153477
.ly	131	59		2253	2377
splunkhvf.froth	192.168.247.	217.182.16	ws001.coinhive.	153477	153477
.ly	131	4.14	com	2269	2273

splunkhwhf.froth .ly	192.168.247. 131	37.187.165. 41	ws005.coinhive. com	153477 2286	153477 2298
splunkhwhf.froth .ly	192.168.247. 131	37.187.166. 108	ws011.coinhive. com	153477 2283	153477 2285
splunkhwhf.froth .ly	192.168.247. 131	37.187.167. 21	ws014.coinhive. com	153477 2254	153477 2267
splunkhwhf.froth .ly	192.168.247. 131	37.187.167. 47	ws019.coinhive. com	153477 2317	153477 3920

Now tell splunk to calculate the duration

```
index="client3" source="ciscovmflowdata"
```

```
| search coinhive.com
```

```
| eval fss = tonumber(fss), fes = tonumber(fes)
```

```
| stats
```

```
  min(fss) AS starttime
```

```
  max(fes) AS endtime
```

```
| eval timetaken = endtime - starttime
```

```
| table timetaken
```

timetaken
1667

The output gives **1667 seconds**

14.3 Leaked AWS keys

Finding Leaked AWS keys

14.3.1 IAM access w most failures

14.3.1 Determining the IAM access key with the most distinct failures

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which IAM access key is responsible for the highest number of unique errors when trying to access IAM resources.

Looking into distinct `errorMessage` as `errorCode` is too broad. Ofcourse since this is AWS `sourcetype:aws:cloudtrail` since im looking for distinct data `stats dc()` or `| stats dc(errorMessage) by userIdentity.accessKeyId`.

Start broad to explore AWS log data:

`index="client3" sourcetype=*aws* *IAM*` use this query and look through the "fields" to find `userIdentity.accessKeyId` and verify what sourcetype I need. **Aws:cloudtrail**

Now to narrow to focus on IAM user events with a `userIdentity.accessKeyId`:

`index="client3" sourcetype="aws:cloudtrail" user_type="IAMUser" "userIdentity.accessKeyId"="*" 5370 total events, still too many to review`

Now narrow down to error events only

`index="client3" sourcetype="aws:cloudtrail" user_type="IAMUser" "userIdentity.accessKeyId"="*" errorCode!="success"`

(!= is short hand for NOT)

Dropping to 1035 events still too many

So Zero in on IAM-specific API calls

`index="client3" sourcetype="aws:cloudtrail" user_type="IAMUser" "userIdentity.accessKeyId"="*" errorCode!="success" eventSource="iam.amazonaws.com"`

12 events

Now filter down to `errorMessage` since `errorCode` is too broad

`index="client3" sourcetype="aws:cloudtrail" user_type="IAMUser" "userIdentity.accessKeyId"="*" errorCode!="success" eventSource="iam.amazonaws.com" | stats dc(errorMessage) as errors by userIdentity.accessKeyId | sort -errors`

Access key **AKIAJOGCDXJ5NW5PXUPA** had the most failures

14.3.2 Amazon support ID

14.3.2 Identifying the Amazon support ID associated with the compromised AWS access key

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

After Bud's AWS keys were leaked to a public repo, AWS assigned a support case ID when notifying him of the breach.

I want to figure out what support case ID AWS assigned him.

Looking into email traffic so `stream:smtp` and servers host name is `matar`.

Ill start broad and narrow down to make sure nothing is missed

`index="client3" sourcetype=stream:smtp host=matar`

879 logs, reviewed for some basic information. Did find one interesting email about info being bought

8/20/18

3:19:35.131 PM

subject: Fw: All your datas belong to us

M

{ [-]

attach_content_decoded_md5_hash: [[-]
807d17ca8c7f400be030ac992cec5b26

]

attach_content_md5_hash: [[+]

]

attach_disposition: [[+]

]

attach_filename: [[+]

]

attach_size: [[+]

]

attach_size_decoded: [[+]

]

attach_transfer_encoding: [[+]

```
]
attach_type: [ [+]
]
bytes: 133287
bytes_in: 133250
bytes_out: 37
content: [ [+]
]
content_body: [ [+]
]
content_transfer_encoding: [ [+]
]
content_type: multipart/related;
    boundary="_004_MWHPR17MB124780ACE6F28E61609F84EABF2B0MWHPR17MB1247namprd17.prod.outlook.com";
    type="multipart/alternative"
date: Fri, 14 Sep 2018 11:25:42 +0000
dest_ip: 172.31.38.181
dest_mac: 06:6A:51:FA:0A:B0
dest_port: 25
endtime: 2018-08-20T15:19:35.131703Z
file_type: [ [+]
]
flow_id: b064f780-d3a0-4841-a5bd-b07a4937d436
mime_type: multipart/related
mime_version: 1.0
msg_id:
<MWHPR17MB124780ACE6F28E61609F84EABF2B0@MWHPR17MB1247.namprd17.prod.outlook.com>
protocol_stack: ip:tcp:smtp
received_by_name: MWHPR17MB1247.namprd17.prod.outlook.com
received_date: [ [+]
]
received_from_name: MWHPR17MB1247.namprd17.prod.outlook.com
received_server_agent: [fe80::74b3:5d25:4f24:72ba%3]
received_with: mapi
receiver: [ [-]
    Bud Stoll <bstoll@froth.ly>
    Billy Tun <btun@froth.ly>
    Jeremiah Wortoski <jwortoski@froth.ly>
]
receiver_alias: [ [+]
]
receiver_email: [ [+]
```

```

]
receiver_type: [ [+]
]
reply_time: 4676
request_time: 349994
response_code: 250
response_time: 0
sender: Grace Hoppy <ghoppy@froth.ly>
sender_alias: Grace Hoppy
sender_email: ghoppy@froth.ly
server_response: 250 2.0.0 Ok: queued as 6C7831794E8
src_ip: 104.47.38.43
src_mac: 06:E3:CC:18:AA:33
src_port: 1920
subject: Fw: All your datas belong to us
time_taken: 354670
timestamp: 2018-08-20T15:19:34.777033Z
transport: tcp
}

```

But for now i need to look into the support case so narrow results using keywords

```

index=client3 sourcetype=stream:smtp host=matar (AWS OR Amazon) (compromised OR notification OR
"support case")
| spath content_body
| spath sender_email
| spath "receiver_email{}"
| rex "New Support case: (?<case_id>\\d+)"
| table _time case_id sender_email receiver_email{}

```

_time	case_id	sender_email	receiver_email{}
2018-08-20 09:16:55.260	52443296 01	no-reply-aws@amazon.com	bstoll@froth.ly
2018-08-20 09:16:18.990		security@mail.gitguardian.com	bstoll@froth.ly

I get case ID 5244329601.

14.3.3 Leaked AWS key pair

14.3.3 Finding the leaked AWS key pair

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the secret access key associated with the AWS key that was exposed in the external repository.

AWS access keys act like a username (Access Key ID) and password (Secret Access Key). Any key starting with AKIA is to be treated as highly sensitive.

Again starting with email traffic

```
index=client3 sourcetype=stream:smtp host=matar
```

Now narrow down looking for remain containing AKIA

```
index=client3 sourcetype=stream:smtp host=matar
```

```
| spath "content_body{}" | search "content_body{}"="*AWS Access Key AKIA*"
```

```
| spath sender_email
```

```
| spath "receiver_email{}"
```

```
| spath subject
```

I get one event at 8/20/18 9:16:55.260 AM

Subject: Your AWS account 622676721278 is compromised

Severity: Urgent

Correspondence: Dear AWS Customer,

Your AWS Account is compromised! Please review the following notice and take immediate action to secure your account.

Your security is important to us. We have become aware that the AWS Access Key

AKIAJOGCDXJ5NW5PXUPA (belonging to IAM user "web_admin") along with the corresponding Secret Key is publicly available online at

https://github.com/FrothlyBeers/BrewingIoT/blob/e4a98cc997de12bb7a59f18aea207a28bcec566c/MyDocuments/aws_credentials.bak.

This poses a security risk to your account and other users, could lead to excessive charges from unauthorized activity or abuse, and violates the AWS Customer Agreement.

Please delete the exposed credentials from your AWS account by using the instructions below and take steps to prevent any new credentials from being published in this manner again. Unfortunately, deleting the keys from the public website and/or disabling them is NOT sufficient to secure your account.

To additionally protect your account from excessive charges, we have temporarily limited your ability to create some AWS resources. Please note that this does not make your account secure, it just partially limits the unauthorized usage for which you could be charged.

Detailed instructions are included below for your convenience.

CHECK FOR UNAUTHORIZED USAGE

We strongly encourage you to immediately review your AWS account for any unauthorized A

WS usage, suspect running instances, or inappropriate IAM users and policies. To check the usage, please log into your AWS Management Console and go to each service page to see what resources are being used. Please pay special attention to the running EC2 instances and IAM users, roles, and groups. You can also check for any unexpected usage on the "Bills" page in the Billing console.

<https://console.aws.amazon.com/billing/home#/bill>

Please keep in mind that unauthorized usage can occur in any region and that in your console you only see one region at a time. To switch between regions, you can use the dropdown in the top-right corner of the console screen.

DELETE THE KEY (ROOT ACCOUNT)

If you are not using the access key, you can simply delete it. To delete the exposed key, visit the "Security Credentials" page here: https://console.aws.amazon.com/iam/home#security_credential. Your keys will be listed in the "Access Keys" section.

DELETE THE KEY (IAM USERS)

Navigate to your IAM Users list in the AWS Management Console, here:

<https://console.aws.amazon.com/iam/home#users>. Please select the IAM user identified above. Click on the "User Actions" drop-down menu and then click "Manage Access Keys" to show that user's active Access Keys. Click "Delete" next to the access key identified above.

ROTATE THE KEY

If your application uses the access key, you need to replace the exposed key with a new one. To do this, first create a second key (at that point both keys will be active) and modify your application to use the new key. Then disable (but do not delete) the first key. If there are any problems with your application, you can make the first key active again. When your application is fully functional with the first key inactive, please delete the first key.

Please follow the Best Practices of Managing your Access Keys at
<http://docs.aws.amazon.com/general/latest/gr/aws-access-keys-best-practices.html>.

If you have any additional questions or concerns regarding this notification, please reach out to us via the AWS Support Center: <https://aws.amazon.com/support>.

=====

To contact us again about this issue, please use the following link to enter your correspondence or attach any files you think would be useful:

<https://console.aws.amazon.com/support/home?#/case/?caseId=5244329601&displayId=5244329601&language=en>

(If you will connect by federation, log in before following the link.)

Sincerely,
The Amazon Web Services Team

*Please note: this e-mail was sent from an address that cannot accept incoming e-mail. Please use the appropriate link above if you need to contact us again about this same issue.

Amazon Web Services, Inc. is an affiliate of Amazon.com, Inc. Amazon.com is a registered trademark of Amazon.com, Inc. or its affiliates.

.

Visiting the github referenced above I find this:

[web_admin]

aws_access_key_id = AKIAJOGCDXJ5NW5PXUPA

aws_secret_access_key = Bx8/gTsYC98T0oWiFhpm�ROqhELPtXJSR9vFPNGk

region = us-west-1

14.3.4 Resource launched by leaked key

14.3.4 Identifying the name of the resource launched using the leaked key

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the name of the resource that the attacker tried to create a key for using the compromised access key.

Since this pivot requires a look in, AWS itself, I will be looking back at CloudTrail logs. These are JSON structured. Looking for the API method `CreateAccessKey`, under `requestParameters.userName`.

And I know the compromised user and key so I'll be able to narrow down my query to look into that.

Stage the query and filter for Access Management (IAM):

```
index=client3 sourcetype="aws:cloudtrail" eventSource="iam.amazonaws.com"
```

8 events listed

Now narrow down to the access key

```
index=client3 sourcetype="aws:cloudtrail" eventSource="iam.amazonaws.com"
userIdentity.accessKeyId="AKIAJOGCDXJ5NW5PXUPA"
```

6 results

Now narrow down to access key creation attempts

```
index=client3 sourcetype="aws:cloudtrail" eventSource="iam.amazonaws.com"
userIdentity.accessKeyId="AKIAJOGCDXJ5NW5PXUPA" eventName="CreateAccessKey"
```

1 event

8/20/18

9:16:12.000 AM

{ [-]

awsRegion: us-east-1

errorCode: AccessDenied

errorMessage: User: arn:aws:iam::622676721278:user/web_admin is not authorized to perform:

iam:CreateAccessKey on resource: user nullweb_admin

eventID: 7c62eeba-2159-41b1-ab8a-26ac6669bd80

eventName: CreateAccessKey

eventSource: iam.amazonaws.com

eventTime: 2018-08-20T09:16:12Z

eventType: AwsApiCall

```
eventVersion: 1.02
recipientAccountId: 622676721278
requestID: 1377f1d2-9093-11e8-a22b-759b04dac456
requestParameters: null
responseElements: null
sourceIPAddress: 35.153.154.221
userAgent: Boto3/1.7.44 Python/2.7.12 Linux/4.4.0-1063-aws Botocore/1.10.44
userIdentity: { [+]  
}
```

Extract the target IAM user:

```
index=client3 sourcetype="aws:cloudtrail" eventSource="iam.amazonaws.com"  
userIdentity.accessKeyId=AKIAJOGCDXJ5NW5PXUPA eventName="CreateAccessKey"  
| fields eventName, errorCode, errorMessage, user_name
```

Attacker tried to create new user: **nullweb_admin**

14.3.5 Application used for unauthorized request

14.3.5 Determining the application responsible for the unauthorized request using the leaked key

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the complete user agent string from the application that attempted to describe an account using the leaked key.

This again will utilize aws:cloudtrail. I AKIAJOGCDXJ5NW5PXUPA is the leaked key. Looking for `eventName` maybe `DescribeAccountAttributes` and the `userAgent`.

Stage the query:

```
index=client3 sourcetype="aws:cloudtrail" userIdentity.accessKeyId=AKIAJOGCDXJ5NW5PXUPA
```

Basic query provides 9 events

Narrow down relevant AWS API action

```
index=client3 sourcetype="aws:cloudtrail" userIdentity.accessKeyId=AKIAJOGCDXJ5NW5PXUPA  
eventName=DescribeAccountAttributes
```

1 result

The user agent found:

`userAgent: ElasticWolf/5.1.6`

```
8/20/18      { [-]
9:27:06.000    awsRegion: us-east-1
AM            errorCode: Client.UnauthorizedOperation

              errorMessage: You are not authorized to perform
this operation.

              eventID: c077df0d-2435-4152-9127-09e579dd1fb2
              eventName: DescribeAccountAttributes
              eventSource: ec2.amazonaws.com
              eventTime: 2018-08-20T09:27:06Z
              eventType: AwsApiCall
              eventVersion: 1.05
              recipientAccountId: 622676721278
              requestId: f94dfb04-2d7b-40a8-b3cc-3664b9463db8
              requestParameters: { [+]
}
              responseElements: null
              sourceIPAddress: 82.102.18.111
              userAgent: ElasticWolf/5.1.6
              userIdentity: { [+]
```

14.3.6 OS of Ubuntu instance

14.3.6 Classifying the OS version of a launched Ubuntu instance

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the Ubuntu version codename used in the adversary's first launch attempt with the compromised IAM account.

This again uses `aws:cloudtrail` this time looking into `RunInstances`.

Beginning query:

```
index=client3 sourcetype="aws:cloudtrail" eventName=RunInstances
```

Queries 582 events

Since i need to know the first instance only no need to narrow down to much instead:

```
index=client3 sourcetype="aws:cloudtrail" eventName=RunInstances | sort _time  
| head 1
```

```
8/20/18      { [-]
9:16:22.000   awsRegion: us-east-1
AM           errorCode: Client.InstanceLimitExceeded

           errorMessage: You have requested more instances (1)
           than your current instance limit of 0 allows for the
           specified instance type. Please visit
           http://aws.amazon.com/contact-us/ec2-request to request an
           adjustment to this limit.

           eventID: bc4f6111-cdf1-49b3-8291-62c9f11779b2

           eventName: RunInstances

           eventSource: ec2.amazonaws.com

           eventTime: 2018-08-20T09:16:22Z

           eventType: AwsApiCall

           eventVersion: 1.05

           recipientAccountId: 622676721278

           requestID: 9525d6c3-bffe-4df3-8b89-ef45cdc9e252

           requestParameters: { [+]

           }

           responseElements: null

           sourceIPAddress: 139.198.18.205

           userAgent: Boto3/1.7.42 Python/2.7.12
           Linux/4.4.0-131-generic Botocore/1.10.42

           userIdentity: { [+]

           }

}
```

[Show as raw text](#)

host = splunk.froth.ly

index = **client3**

source =

<s3://cloudtrail-622676721278/AWSLogs/622676721278/CloudTrail/>

```
us-east-1/2018/07/26/622676721278_CloudTrail_us-east-1_20180726T0520Z_SVmaSY7H67epleut.json.gz
```

Now extract the AMI ID:

_time	userIdentity.userName	image_id	requestParameters.instanceType	awsRegion
2018-08-20 09:16:22	web_admin	ami-41 e0b93 b	x1.32xlarge	us-east-1

Using Ubuntu EC2 AMI Locator

Ubuntu version

xenial

14.3.7 Avg length of unique subdomains

14.3.7 Calculating the average length of unique subdomains

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Frothly uses Amazon Route 53 for DNS.

I want to figure out the average character length of unique third-level subdomains queried for brewertalk[.]com in Route 53.

(e.g., The third-level subdomain for earth.solarsystem.universe.com is solarsystem.)

You'll also want to round the answer to two decimal places (e.g., 42.06).

This query will utilize `aws:cloudwatchlogs` and a source log: `source="lambda:dns"`. This will require `ut_parse_extended()` to split a domain.

Stage the query:

```
index="client3" sourcetype="aws:cloudwatchlogs" source="lambda:dns"
115145 events
```

Organize by [brewertalk.com](#):

```
index="client3" sourcetype="aws:cloudwatchlogs" source="lambda:dns" | eval domain=replace(_raw, ".*" ([^
]+\\.brewertalk\\.com) \\.*, "\1")
```

Then split the domains and pipe them into a table

```
index="client3" sourcetype="aws:cloudwatchlogs" source="lambda:dns" | eval domain=replace(_raw, ".*" ([^
]+\\.brewertalk\\.com) \\.*, "\1") | `ut_parse_extended(domain, "list=brewertalk")` | table _time, host, domain,
ut_subdomain_level_1, ut_domain, ut_tld | rex field=_raw
"(?<ut_subdomain_level_1>[a-zA-Z0-9\-\+])\.(?<ut_domain>brewertalk)\.(?<ut_tld>com)"
```

Now dedup and reorganize for better clarity:

```
index=client3 source="lambda:dns" brewertalk.com
| rex field=_raw "(?<ut_subdomain_level_1>[a-zA-Z0-9\-\+])\.(?<ut_domain>brewertalk)\.(?<ut_tld>com)"
| dedup ut_subdomain_level_1
| table _raw, ut_subdomain_level_1, ut_domain, ut_tld
```

Now measure the length of each subdomain:

```
index=client3 sourcetype="aws:cloudwatchlogs" source="lambda:dns"
| rex field=_raw "(?<domain>[a-zA-Z0-9\-.]+\.\brewertalk\.com)"
| rex field=_raw "(?<ut_subdomain_level_1>[a-zA-Z0-9\-.]+\.\(?<ut_domain>brewertalk\)\. (?<ut_tld>com)"
| dedup ut_subdomain_level_1
| eval length=len(ut_subdomain_level_1)
| table domain, ut_subdomain_level_1, ut_domain, ut_tld, length
```

Then calculate the average subdomain:

```
| stats avg(length) AS avglength
| eval avglength=round(avglength, 2)
```

8.10 is the average length

14.4 A compromised endpoint

14.4.1 User agent string for malicious upload

14.4.1 Unveiling the user agent string associated with the malicious upload

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the full user agent string identifying the client that uploaded the malicious shortcut file (.lnk) to OneDrive.

I'll be looking into fields like `Workload="OneDrive"` or `Operation="FileUploaded"` to reduce noise.

First a broad query to identify the sourcetypes:

```
index="client3" onedrive lnk  
| stats count by sourcetype
```

sourcetype	count
ms:o365:management	24
o365:management:activity	24
symantec:ep:risk:file	1

Since I have 2 possible sourcetypes I'll dig into each to find which has the fields I'm looking for.

First

```
index="client3" sourcetype="*o365:management*" lnk
```

```
Workload: OneDrive
```

I'll narrow my results using this field:

```
index="client3" sourcetype="*o365:management*" Workload="OneDrive"
```

Narrow the results to upload actions only

```
Operation: FileUploaded
```

```
index="client3" sourcetype="*o365:management*" Workload="OneDrive" Operation="FileUploaded"
```

Still gives 14 results

So ill narrow down again to lmk (the file we are looking for)

```
index="client3" sourcetype="*o365:management*" Workload="OneDrive" Operation="FileUploaded" lmk
```

2 results

Now ill pipe the result into a table for clean viewing

```
index="client3" sourcetype="*o365:management*" Workload="OneDrive" Operation="FileUploaded" lmk | table _time, sourcetype, src_ip, user, object, SourceFileName, UserAgent
```

_time	sourcetype	src_ip	user	object	SourceFileName	UserAgent
2018-08-20 09:57:33	o365:management:activity	104.20 7.83.63	bgist@fr oth.ly	BRUCE	BRUCE	Mozilla/5.0
				CE	BIRTHDAY	(X11; U;
				BIRTHDAY	HAPPY	Linux i686;
				DAY	HOUR	ko-KP; rv:
				HA	PICS.Ink	19.1br)
				PPY		Gecko/2013
				HO		0508
				UR		Fedora/1.9.
				PIC		1-2.5.rs3.0
				S.Ink		NaenaraBrowser/3.5b4
2018-08-20 09:57:33	ms:o365:management	104.20 7.83.63	bgist@fr oth.ly	BRUCE	BRUCE	Mozilla/5.0
				CE	BIRTHDAY	(X11; U;
				BIRTHDAY	HAPPY	Linux i686;
				DAY	HOUR	ko-KP; rv:
				HA	PICS.Ink	19.1br)
				PPY		Gecko/2013
						0508

HO	Fedora/1.9.
UR	1-2.5.rs3.0
PIC	NaenaraBro
S.In	wser/3.5b4
k	

I get user agent string:

Mozilla/5.0 (X11; U; Linux i686; ko-KP; rv: 19.1br) Gecko/20130508 Fedora/1.9.1-2.5.rs3.0
NaenaraBrowser/3.5b4

Doing side research

① Mozilla/5.0

Historically, most browsers identify themselves starting with Mozilla.
Nowadays, it's mostly for compatibility, not a literal claim that it's Firefox.
5.0 usually indicates a modern browser standard.

② (X11; U; Linux i686; ko-KP; rv: 19.1br)

This is the platform / OS / locale info:

X11 → The browser is running on Unix/Linux graphical environment (common in desktops)

U → Usually means "security level: strong" (from older Netscape conventions)

Linux i686 → The OS is Linux on a 32-bit Intel CPU

ko-KP → Locale / language code

ko → Korean

KP → North Korea

→ So system language is Korean (North Korea)

rv:19.1br → Rendering engine version (Gecko engine)

19.1br suggests a custom or branch version, not standard Firefox

③ Gecko/20130508

Gecko = Mozilla's browser engine (used in Firefox)

20130508 = build date of the engine (May 8, 2013)

So this is a very old Gecko version, but likely modified

④ Fedora/1.9.1-2.5.rs3.

Suggests underlying Linux distribution or packaging

Fedora = based on Fedora Linux

1.9.1-2.5.rs3.0 = custom build version for their environment

5 NaenaraBrowser/3.5b4

This is the actual browser name:

Naenara Browser = North Korea's custom browser, made to run on Red Star OS (North Korean Linux derivative)

3.5b4 = version number (beta 4 of 3.5)

✓ TL;DR

OS: Linux i686 (32-bit), likely Red Star OS / Fedora-based

Browser: Naenara Browser 3.5 beta 4

Engine: Gecko (like Firefox)

Locale: Korean (North Korea)

Build: custom dated 2013

💡 Extra context:

The Naenara Browser is North Korea's standard browser, heavily modified for government monitoring and limited Internet access. Seeing this user agent almost always indicates a user inside the North Korean intranet (Kwangmyong).

14.4.2 IP of logged-in expired account

14.4.2 Revealing the IP associated with successful logins using an expired account

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the external client IP address that successfully logged into Frothy using an expired account.

An "expired account" or "expired password" doesn't mean the account is gone – it just means credentials are no longer valid until refreshed. Logs might not say "this account expired" directly. Instead, they hint at it with error messages like Invalid password, entered expired password.

This will involve Azure AD sign-in logs `sourcetype="ms:aad:signin"` and critical fields such as `ResultStatus` (success or failure), `failureReason`, `userDisplayName`, and `ClientIP`.

Starting broad:

```
index="client3" sourcetype="ms:aad:signin" expired
```

Only on event parsed

8/20/18

11:43:14.799 AM

```
{ [-]
```

```
  appDisplayName: Microsoft Office 365 Portal
```

```
  appId: 00000006-0000-0ff1-ce00-000000000000
```

```
  correlationId: 3f2947b4-24fd-4989-9a3b-452a9f2525f1
```

```
  dataSource: 1
```

```
  deviceInformation: ;MacOs;Chrome 67.0.3396;
```

```
  failureReason: Invalid password, entered expired password.
```

```
  geoCoordinates: { [+]
```

```
  }
```

```
  id: af845528-2298-47ba-8b52-109789240600
```

```
  ipAddress: 199.66.91.253
```

```
  location: { [-]
```

```
    city: Clarkson Village
```

```
    country: CA
```

```
    state: Ontario
```

```
  }
```

```
  loginStatus: Failure
```

```
  mfaAuthDetail: null
```

```
  mfaAuthMethod: null
```

```
mfaRequired: false
mfaResult: null
signinDateTime: 2018-08-20T11:43:14.7994359Z
signinDateTimelnMillis: 1534765394799
signinErrorCode: 50055
userDisplayName: Kevin Lagerfield
userId: 90aaf22f-6e60-48fb-9cca-5fbd8984225e
userPrincipalName: klagerfield@froth.ly
}
```

Now to dial in my query on Kevin Lagerfield

```
index="client3" sourcetype="ms:aad:signin" userDisplayName="Kevin Lagerfield" loginStatus=Success
```

8 events

Checking to see if any came after 8/20/18 11:43:14.799 AM

All 8 event happens post the failed login

The first event after is

8/20/18

11:43:32.927 AM

```
{ [-]
  appDisplayName: Microsoft Office 365 Portal
  appId: 00000006-0000-0ff1-ce00-000000000000
  correlationId: 3f2947b4-24fd-4989-9a3b-452a9f2525f1
  dataSource: 1
  deviceInformation: ;MacOs;Chrome 67.0.3396;
  failureReason: null
  geoCoordinates: { [+]}
  id: 91e2dfd4-b7d4-41b0-b50c-a247ad009400
  ipAddress: 199.66.91.253
  location: { [-]
    city: Clarkson Village
    country: CA
    state: Ontario
  }
  loginStatus: Success
  mfaAuthDetail: null
  mfaAuthMethod: null
  mfaRequired: false
  mfaResult: null
}
```

signinDateTime: 2018-08-20T11:43:32.9277819Z
signinDateTimelnMillis: 1534765412927
signinErrorCode: 0
userDisplayName: Kevin Lagerfield
userId: 90aaf22f-6e60-48fb-9cca-5fbd8984225e
userPrincipalName: klagerfield@froth.ly

The IP address is external 199.66.91.253

Querying the chronological order of events

```
index="client3" sourcetype="ms:aad:*" (userDisplayName="Kevin Lagerfield" OR  
user_id="kevin.lagerfield@froth.ly")  
| sort _time
```

This log appears before the successful login.

8/20/18

11:43:24.052 AM

{ [-]

appDisplayName: Microsoft Office 365 Portal
appId: 00000006-0000-0ff1-ce00-000000000000
correlationId: 3f2947b4-24fd-4989-9a3b-452a9f2525f1
dataSource: 1

deviceInformation: ;MacOs;Chrome 67.0.3396;

failureReason: This error occurred due to 'Keep me signed in' interrupt when the user was signing-in. Open
a support ticket with Correlation ID, Request ID, and Error code to get more details.

geoCoordinates: { [+]
}

id: c56d8417-f978-4453-a0aa-9c0c131f6c00

ipAddress: 199.66.91.253

location: { [+]

}

loginStatus: Failure

mfaAuthDetail: null

mfaAuthMethod: null

mfaRequired: false

mfaResult: null

signinDateTime: 2018-08-20T11:43:24.0525703Z

signinDateTimelnMillis: 1534765404052

signinErrorCode: 50140

userDisplayName: Kevin Lagerfield

userId: 90aaf22f-6e60-48fb-9cca-5fbd8984225e

```
    userPrincipalName: klagerfield@froth.ly  
  }
```

Show as raw text

14.4.3 Discovery date of malicious macro

14.4.3 Using OSINT to establish the discovery date of the malicious macro

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the discovery date of the malware found in the macro-enabled document (according to Symantec).

This will utilize WinEventLog:Application sourcetype which captures logs from Windows applications, including antivirus products like Symantec.

First off searching broadly for macro-related artifacts

```
index="client3" *macro* *Malware Alert Text.txt*
```

```
8/20/18
```

```
9:55:14.436 AM
```

```
{ [-]
```

```
  attach_content_decoded_md5_hash: [ [+]
```

```
]
```

```
  attach_content_md5_hash: [ [+]
```

```
]
```

```
  attach_disposition: [ [-]
```

```
    attachment
```

```
]
```

```
  attach_filename: [ [-]
```

```
    Malware Alert Text.txt
```

```
]
```

```
  attach_size: [ [-]
```

```
    272
```

```
]
```

```
  attach_size_decoded: [ [-]
```

```
    197
```

```
]
```

```
  attach_transfer_encoding: [ [-]
```

```
    base64
```

```
]
```

```
  attach_type: [ [-]
```

```
    application/octet-stream
```

```
]
```

```
bytes: 5539
```


bytes_in: 5502
bytes_out: 37
content: [[+]
]
content_body: [[-]
--_002_34d01f54b269459296334098605e3933DM6PR17MB2139namprd17pr_

Here is a financial model we can use for FY2019 planning. For the workshee=
t to operate properly, you will need to enable macros.

Thanks,Bruce =

.
]
content_transfer_encoding: quoted-printable
content_type: multipart/mixed;
boundary="_002_34d01f54b269459296334098605e3933DM6PR17MB2139namprd17pr_"
date: Mon, 15 Sep 2018 01:21:12 +0000
dest_ip: 172.31.38.181
dest_mac: 06:6A:51:FA:0A:B0
dest_port: 25
endtime: 2018-08-20T09:55:14.436835Z
file_type: [[+]
]
flow_id: 0c9748fa-4d85-4c7c-9042-07edafbe8d35
mime_type: multipart/mixed
mime_version: 1.0
msg_id:
<34d01f54-b269-4592-9633-4098605e3933@DM6PR17MB2139.namprd17.prod.outlook.com>
protocol_stack: ip:tcp:smtp
received_by_name: [[+]
]
received_date: [[+]
]
received_from_name: [[-]
SmtpServer.Submit
DM6PR17MB2186.namprd17.prod.outlook.com
]
received_server_agent: [fe80::d97d:3d05:b533:b293%3]
received_with: [[-]
Microsoft
mapi

]

receiver: [[-]

Bud Stoll <bstoll@froth.ly>

Fyodor Malteskesko <fyodor@froth.ly>

Grace

Hoppy <ghoppy@froth.ly>

Al Bungstein <abungstein@froth.ly>

]

receiver_alias: [[-]

Bud Stoll

Fyodor Malteskesko

Grace

Hoppy

Al Bungstein

]

receiver_email: [[-]

bstoll@froth.ly

fyodor@froth.ly

ghoppy@froth.ly

abungstein@froth.ly

]

receiver_type: [[-]

TO

TO

TO

TO

]

reply_time: 3284

request_time: 229

response_code: 250

response_time: 0

sender: Bruce Gist <bgist@froth.ly>

sender_alias: Bruce Gist

sender_email: bgist@froth.ly

server_response: 250 2.0.0 Ok: queued as 5204F1794E8

src_ip: 104.47.40.200

src_mac: 06:E3:CC:18:AA:33

src_port: 31712

subject: Draft Financial Plan for Brewery FY2019

time_taken: 3513

timestamp: 2018-08-20T09:55:14.433322Z

transport: tcp

}

Now decode base64

index=client3 sourcetype=stream:smtp *alert* subject=*

| table _time subject attach_transfer_encoding{} content{} attach_content_decoded_md5_hash{}

I get code:

TWFsd2FyZSB3YXMgZGV0ZWN0ZWQgaW4gb25lIG9yIG1vcmlUgYXR0YWNobWVudHMgaW5jbHVkZWQg
d2l0aCB0aGlzIGVtYWlsIG1lc3NhZ2UuIA0KQWN0aW9uOiBBbGwgYXR0YWNobWVudHMgaGF2ZSBi
ZWVuIHJlbW92ZWQudQpGcm90aGx5LUJyZXdlcnktRmluYW5jaWFsLVBsYW5uaW5nLUZZMjAxOS1E
cmFmdC54bHNtCSBXOTdNLkVtcHN0YWdlIDQo=

--_002_34d01f54b269459296334098605e3933DM6PR17MB2139namprd17pr--

.

Translates to:

Malware was detected in one or more attachments included with this email message.

Action: All attachments have been removed.

Frothly-Brewery-Financial-Planning-FY2019-Draft.xlsm

W97M.Empstage

ÓM·áÝ5 Ü~xçÝ½ë}øÓß:Ó--÷}Ã3£Ñ×³Ûjý ©©Ý{!

Now search internally for malware name

index=client3 sourcetype=xmlwineventlog source="WinEventLog:Microsoft-Windows-Sysmon/Operational"

xlsm

| sort by +_time

Time	Event
------	-------

8/20/18

9:55:52.000

AM

```
<Event
xmlns='http://schemas.microsoft.com/win/2004
/08/events/event'><System><Provider
Name='Microsoft-Windows-Sysmon'
Guid='{5770385F-C22A-43E0-BF4C-06F5698FFBD9}'
/><EventID>11</EventID><Version>2</Version>
<Level>4</Level><Task>11</Task><Opcode>0</Op
code><Keywords>0x8000000000000000</Keywords>
<TimeCreated
SystemTime='2018-08-20T09:55:52.450060800Z' /
><EventRecordID>36035</EventRecordID><Correl
ation/><Execution ProcessID='3516'
ThreadID='5172' /><Channel>Microsoft-Windows-
Sysmon/Operational</Channel><Computer>BGIST-
L.froth.ly</Computer><Security
UserID='S-1-5-18' /></System><EventData><Data
Name='UtcTime'>2018-08-20
09:55:52.449</Data><Data
Name='ProcessGuid'>{EBF7A186-B00B-5B58-0000-
0010CC954200}</Data><Data
Name='ProcessId'>10096</Data><Data
Name='Image'>C:\Program
Files\WindowsApps\microsoft.windowscommunica
tionsapps_16005.10228.20127.0_x64__8wekyb3d8
bbwe\HxTsr.exe</Data><Data
Name='TargetFilename'>C:\Users\BruceGist\AppData\Local\Packages\microsoft.windowscommuni
cationsapps_8wekyb3d8bbwe\LocalState\Files\S
0\3\Frothly-Brewery-Financial-Planning-FY201
9-Draft[66].xlsm</Data><Data
Name='CreationUtcTime'>2018-08-20
09:55:52.449</Data></EventData></Event>
```

host = BGIST-L

index = client3

		source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = xmlwineventlog
8/20/18 9:55:54.000 AM	<pre> <Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'> <System> <Provider Name='Microsoft-Windows-Sysmon' Guid='{5770385F-C22A-43E0-BF4C-06F5698FFBD9}' /> <EventID>11</EventID> <Version>2</Version> <Level>4</Level> <Task>11</Task> <Opcode>0</Opcode> <Keywords>0x8000000000000000</Keywords> <TimeCreated SystemTime='2018-08-20T09:55:54.785936300Z' /> <EventRecordID>36036</EventRecordID> <Correlation /> <Execution ProcessID='3516' ThreadID='5172' /> <Channel>Microsoft-Windows-Sysmon/Operational</Channel> <Computer>BGIST-L.froth.ly</Computer> <Security UserID='S-1-5-18' /> </System> <EventData> <Data Name='UtcTime'>2018-08-20 09:55:54.785</Data> <Data Name='ProcessGuid'>{EBF7A186-AB12-5B58-0000-0010EB030000}</Data> <Data Name='ProcessId'>4</Data> <Data Name='Image'>System</Data> <Data Name='TargetFilename'>C:\ProgramData\Symantec\Symantec Endpoint Protection\14.2.760.0000.105\SRTSP\Quarantine\AP4D3C539B.xmlsm</Data> <Data Name='CreationUtcTime'>2018-08-20 09:55:54.785</Data> </EventData> </Event> </pre>	host = BGIST-L index = client3

```
source =  
WinEventLog:Microsoft-Windows-Sysmon/Operati  
onal  
sourcetype = xmlwineventlog
```

Malware name: **W97M.Empstage**

Now google search:

W97M.Empstage Symantec.

Discovered 11/12/16

14.4.4 User pw created post-pwning

14.4.4 Uncovering the password of the user created post-compromise

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the password assigned to the newly created user by root on Frothy's local Linux machine.

I'll be looking at Linux Logs via `sourcetype=osquery:results`.

Broad query:

```
index="client3" sourcetype="osquery:results" host="hoth"
```

79,098 events

Narrow results by using Linux user management keywords:

```
index="client3" sourcetype="osquery:results" host="hoth" (useradd OR adduser)
```

2 events

8/20/18

11:24:54.000

AM

```
{ [-]
  action: added
  calendarTime: Mon Aug 20 11:24:54 2018 UTC
  columns: { [-]
    atime: 1534763224
    auid: 4294967295
    btime: 0
    cmdline: "useradd" "-ou" "tomcat7" "-p"
    "ilovedavidverve" "0" "-g" "0" "-M" "-N" "-r" "-s"
    "/bin/bash"
    ctime: 1533402436
    cwd:
    egid: 0
    euid: 0
    gid: 0
    mode: 0100755
    mtime: 1494977854
    owner_gid: 0
    owner_uid: 0
    parent:
    path: /usr/sbin/useradd
    pid: 12815
    status:
    time: 1534764286
    uid: 0
    uptime: 11143
  }
  counter: 0
  decorations: { [+]
  }
```

```
epoch: 0
hostIdentifier: hoth
name: pack_process-monitoring_proc_events
unixTime: 1534764294
}
```

[Show as raw text](#)

Now filter by the root user using UID 0

```
index=client3 sourcetype=osquery:results host=hoth (useradd OR adduser OR passwd) columns.uid=0
```

The look in the interesting fields under `columns.cmdline` look for the -P that gives you the pivot in this case:

```
index=client3 sourcetype=osquery:results host=hoth (useradd OR adduser OR passwd) columns.uid=0
```

```
"columns.cmdline"="\useradd\" \"-ou\" \"tomcat7\" \"-p\" \"ilovedavidverve\" \"0\" \"-g\" \"0\" \"-M\" \"-N\" \"-r\" \"-s\" \"/bin/bash\""
```

To clean it up rex and table the output:

```
index=client3 sourcetype=osquery:results host=hoth (useradd OR adduser OR passwd) columns.uid=0
```

```
"columns.cmdline"="\useradd\" \"-ou\" \"tomcat7\" \"-p\" \"ilovedavidverve\" \"0\" \"-g\" \"0\" \"-M\" \"-N\" \"-r\" \"-s\" \"/bin/bash\"" | rex field=columns.cmdline "passwd\\s+[^\\s]+\\s+(?<password>[^\s]+)" | table _time, host, columns.cmdline, password
```

8/20/18

{ [-]

11:24:54.000

action: added

AM

calendarTime: Mon Aug 20 11:24:54 2018 UTC

columns: { [-]

 atime: 1534763224

 auid: 4294967295

 btime: 0

 cmdline: "useradd" "-ou" "tomcat7" "-p"

"ilovedavidverve" "0" "-g" "0" "-M" "-N" "-r" "-s"
"/bin/bash"

 ctime: 1533402436

 cwd:

 egid: 0

 euid: 0

 gid: 0

 mode: 0100755

 mtime: 1494977854

 owner_gid: 0

 owner_uid: 0

 parent:

 path: /usr/sbin/useradd

 pid: 12815

 status:

 time: 1534764286

 uid: 0

 uptime: 11143

}

counter: 0

decorations: { [+]

}

```
    epoch: 0
    hostIdentifier: hoth
    name: pack_process-monitoring_proc_events
    unixTime: 1534764294
  }
```

8/20/18

11:08:05.000

AM

```
{ [-]
  action: added
  calendarTime: Mon Aug 20 11:08:05 2018 UTC
  columns: { [-]
    atime: 1534763224
    auid: 4294967295
    btime: 0
    cmdline: "useradd" "-ou" "tomcat7" "-p"
"davidverve.com" "0" "-g" "0" "-M" "-N" "-r" "-s"
"/bin/bash"
    ctime: 1533402436
    cwd:
    egid: 117
    euid: 111
    gid: 117
    mode: 0100755
    mtime: 1494977854
    owner_gid: 0
    owner_uid: 0
    parent:
    path: /usr/sbin/useradd
    pid: 5752
    status:
    time: 1534763237
    uid: 111
    uptime: 10094
  }
  counter: 0
  decorations: { [+]
}
```

```
epoch: 0  
hostIdentifier: hoth  
name: pack_process-monitoring_proc_events  
unixTime: 1534763285
```

Ilovedavidverve is the newly created password

14.4.5 User created post-pwning

14.4.5 Retrieving the name of the user created post-compromise

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

After the system was breached, a new user was created.

I want to figure out the name of the new user account that was created.

This will utilize WinEventLog:Security and EventCode 4720. Using mvindex() to pull out the newly created user versus the account that created it.

```
| eval CreatedBy = mvindex(Account_Name, 0)
| eval NewUser = mvindex(Account_Name, 1)
| table _time CreatedBy NewUser host
```

Of course start broad and narrow down:

```
index="client3"
| stats count by host sourcetype
| sort host -count
```

Now narrow down

```
index=client3 sourcetype="WinEventLog" EventCode=4720
```

Provides 1 Event

8/20/18

10:08:17.000 AM

08/19/2018 22:08:17 PM

LogName=Security

SourceName=Microsoft Windows security auditing.

EventCode=4720

EventType=0

Type=Information

ComputerName=FYODOR-L.froth.ly

TaskCategory=User Account Management

OpCode=Info

RecordNumber=277561

Keywords=Audit Success

Message=A user account was created.

Subject:

Security ID: AzureAD\FyodorMalteskesko
Account Name: FyodorMalteskesko
Account Domain: AzureAD
Logon ID: 0x1091C98

New Account:

Security ID: FYODOR-L\svcvnc
Account Name: svcvnc
Account Domain: FYODOR-L

Attributes:

SAM Account Name: svcvnc
Display Name: <value not set>
User Principal Name: -
Home Directory: <value not set>
Home Drive: <value not set>
Script Path: <value not set>
Profile Path: <value not set>
User Workstations: <value not set>
Password Last Set: <never>
Account Expires: <never>
Primary Group ID: 513
Allowed To Delegate To: -
Old UAC Value: 0x0
New UAC Value: 0x15
User Account Control:
 Account Disabled
 'Password Not Required' - Enabled
 'Normal Account' - Enabled
User Parameters: <value not set>
SID History: -
Logon Hours: All

Additional Information:

Privileges -

Now use eval and table to be extract the data

```
| eval CreatedBy = mvindex(Account_Name, 0)
```

```
| eval NewUser = mvindex(Account_Name, 1)
```

```
| table _time CreatedBy NewUser host
```

_time	CreatedBy	NewUser	host
2018-08-20 10:08:17	FyodorMalteskesko	svcvnc	FYODOR -L

svcvnc

Filter again to verify

```
index=client3 "add" "user"
```

```
NOT "Network Connected Devices Auto-Setup"
```

```
NOT ec2-user
```

```
sourcetype!="stream:mysql"
```

svcvnc

14.4.6 1337-port process

14.4.6 Tracing the process responsible for listening on an elite port

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the process ID corresponding to the process listening on a leet-numbered port.

Using `osquery:results` sourcetype again for this pivot as well as ports like `1337` or `31337`.

Broad query:

```
index=client3 sourcetype=osquery:results host=hoth
```

Now look at ports

Add `name=*port*`

9 events

1 stands out:

8/20/18

11:55:34.000 AM

{ [-]

action: added

calendarTime: Mon Aug 20 11:55:34 2018 UTC

columns: { [-]

address: 0.0.0.0

family: 2

fd: 3

net_namespace: 4026531957

path:

pid: 14356

port: 1337

protocol: 6

socket: 254926

}

counter: 5

decorations: { [+]

}

epoch: 0

hostIdentifier: hoth

name: pack_incident-response_listening_ports

unixTime: 1534766134

Now to verify:

```
index=client3 sourcetype=osquery:results host=hoth name=*port* (1337 OR 31337)
```

Same event populates

The process ID or pid:

14356

14.4.7 Search query in URI

14.4.7 Decoding a search query from a URI

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the search string used in the query that came from an external IP address associated with Frothy's mail server.

I need to use DNS tools like `dig` or `host` to identify the actual mail server and its associated IPs.

`o365:management:activity sourcetype prefix 104`

Broad query

`index=client3 sourcetype=o365:management:activity`

`| stats count by src_ip, Operation`

977 events

Narrow the search by prefix:

`index=client3 sourcetype=o365:management:activity`

`| eval prefix=mvindex(split(src_ip,"."),0) . "." . mvindex(split(src_ip,"."),1)`

`| stats count by prefix`

prefix	count
104.207	37
104.209	866
104.238	4
104.47	2
107.77	16

Now looking for the CIDR

`index=client3 sourcetype=o365:management:activity`

```
| stats count by src_ip, Operation
```

2 IP address

```
104.209.132.239      SearchQueryPerformed  4
```

```
104.47.38.254      SearchQueryPerformed  2
```

Now confirm:

```
index=client3 sourcetype=o365:management:activity
```

```
| where cidrmatch("104.47.0.0/16", src_ip)
```

```
| stats count by src_ip, Operation
```

src_ip	Operation	count
104.47.38.254	SearchQueryPerformed	2

104.47.38.254

Now correlate that with New-MailboxSearch to see what the user was looking for

```
index=client3 sourcetype=o365:management:activity Operation="New-MailboxSearch"
```

```
| table src_ip, UserId, Params
```

8/20/18

10:48:28.000 AM

{ [-]

ClientIP: 104.207.83.63:21974

CreationTime: 2018-08-20T10:48:28

ExternalAccess: false

Id: 4f3ca1bb-1e03-4da6-7cf0-08d5f2643ddd

ObjectId:

Operation: New-MailboxSearch

OrganizationId: 225e05a1-5914-4688-a404-7030e60f3143

OrganizationName: frothly.onmicrosoft.com

OriginatingServer: CY4PR17MB1398 (15.20.0973.010)

Parameters: [[-]

{ [-]

Name: Name

Value: SOX

}

{ [-]

```
Name: SearchQuery
```

```
Value: cromdale OR beer OR financial OR secret
```



```
}  
{ [-]  
  Name: AllSourceMailboxes  
  Value: True  
}  
{ [-]  
  Name: EstimateOnly  
  Value: True  
}  
]  
RecordType: 1  
ResultStatus: True  
SessionId:  
UserId: fyodor@froth.ly  
UserKey: 1003BFFDA2E71FF9  
UserType: 2  
Version: 1  
Workload: Exchange
```

The search query is:

```
Name: SearchQuery  
Value: cromdale OR beer OR financial OR secret
```

14.4.8 MD5 of scanning binary

14.4.8 Identifying the MD5 associated with the scanning binary

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the MD5 hash of the file that was downloaded to Fyodor's system and used to perform a network scan at Frothly.

This query will utilize Sysmon `XmlWinEventLog:Microsoft-Windows-Sysmon/Operational` and `stats dc(DestinationPort)`.

Broad query:

```
index=client3 source="WinEventLog:Microsoft-Windows-Sysmon/Operational" host="FYODOR-L"
4126 events
```

Update to extract ports:

```
index=client3 host="FYODOR-L" source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
| rex field=_raw "<Data Name='DestinationPort'>(?(DestinationPort>\d+)</Data>"
| table _time DestinationPort
```

Not refine that query

```
index=client3 host="FYODOR-L" source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
| rex field=_raw "<Data Name='DestinationPort'>(?(DestinationPort>\d+)</Data>"
| rex field=_raw "<Data Name='Image'>(?(Image>[^<]+)</Data>"
| stats dc(DestinationPort) as DestinationPortDistinctCount by host, Image
| sort - DestinationPortDistinctCount
```

host	Image	DestinationPortDistinctCount
FYODOR-L	C:\Windows\Temp\hdoor.exe	10

Filter to that executable and extract its hash

```
index=client3 host="FYODOR-L" source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
| rex field=_raw "<Data Name='Image'>(?(Image>[^<]+)</Data>"
```

```
| search Image="*hdoor.exe"
```

```
| rex field=_raw "<Data Name='Hashes'>(?!<Hashes>[^\<]+)</Data>"
```

```
| table _time Image Hashes
```

2018-08-	C:\Windows\	MD5=586EF56F4D8963DD546163AC31C865D7,SHA256=9992
20	Temp\hdoor.	5199059EE049F7AEDA8904C2F5BDFBA86671FD7A5989BD60
10:43:10	exe	B72F26EF737C

14.4.9 Groups user added to post-pwning

14.4.9 Confirming the groups the user was added to post-compromise

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which user groups were assigned to this user after the compromise of the endpoint. Using WinEventLog:Security, use Event ID 4732 to identify group additions.

Broad query:

```
index=client3 "add" "user"
```

Then refine

```
index=client3 "add" "user" NOT "Network Connected Devices Auto-Setup" NOT ec2-user  
sourcetype!="stream:mysql"
```

Looking for Event ID 4732 – this logs when a user is added to a security so refine again:

```
index=client3 source=WinEventLog:Security EventCode=4732 svcvnc
```

Time	Event
8/20/18 10:08:35.000 AM	<pre>08/19/2018 22:08:35 PM ... 16 lines omitted ... Member: Security ID: FYODOR-L\svcvnc Account Name: - Group: Show all 26 lines host = FYODOR-L index = client3 source = WinEventLog:Security sourcetype = WinEventLog</pre>

```
8/20/18      08/19/2018 22:08:17 PM
10:08:17.000 ... 16 lines omitted ...
AM           Member:
              Security ID:
              FYODOR-L\svcvnc
              Account Name:      -
              Group:
              Show all 26 lines
              host = FYODOR-L
              index = client3
              source = WinEventLog:Security
              sourcetype = WinEventLog
```

Identify that user `svcvnc` pops up from exercise 14.4.5.

Now extract group names from structured fields:

```
index=client3 source=WinEventLog:Security EventCode=4732 svcvnc
| table _time, host, EventCode, Message, Group_Name, member_id
```

_time	host	Event Code	Message	Group_Name	member_id
-------	------	------------	---------	------------	-----------

2018-08-20 10:08:35	FYODOR-L	4732	A member was added to a security-enabled local group. Subject: Security ID: AzureAD\FyodorMalteskesko Account Name: FyodorMalteskesko Account Domain: AzureAD Logon ID: 0x1091C98 Member: Security ID: FYODOR-L\svcvnc Account Name: - Group: Security ID: BUILTIN\Administrators Group Name: Administrators Group Domain: Builtin Additional Information:	Administrators	AzureAD\FyodorMalteskesko FYODOR-L\svcvnc BUILTIN\Administrators
------------------------	----------	------	---	----------------	---

Privileges:

-

2018-08-20 10:08:17	FYODOR-L	4732	A member was added to a security-enabled local group.	Users	AzureAD\FyodorMalteskesko FYODOR-L\svcvnc BUILTIN\Users
------------------------	----------	------	---	-------	---

Subject:

Security ID:

AzureAD\FyodorMalteskesko

Account

Name:

FyodorMalteskesko

Account

Domain:

AzureAD

Logon ID:

0x1091C98

Member:

Security ID:

FYODOR-L\svcvnc

Account

Name:

-

Group:

Security ID:

BUILTIN\Users

Group

Name:

Users

Group

Domain:

Builtin

Additional

Information:

Privileges:

-

User svcnc was added to **Administrators, Users**.

14.4.10 Email address of disabled user

14.4.10 Extracting the disabled user's email address

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which email address belonged to the disabled user and which belonged to the disabler during the attack.

This involves using Azure AD audit logs (ms:aad:audit) initiatedBy object for the actor and targetResources for the target.

First i need to query for Disable account activity:

```
index=client3 sourcetype="ms:aad:audit" activity="Disable account"
```

8/20/18

{ [-]

2:47:12.515 PM

```
    activity: Disable account
    activityDate: 2018-08-20T14:47:12.5151783Z
    activityDateInMillis: 1534776432515
    activityOperationType: Update
    activityResultDescription:
    activityResultStatus: Success
    activityType: User
    actor: { [+]
    }
    actorType: User
    additionalDetails: [ [+]
    ]
    category: Core Directory
    componentOrSource: null
    correlationId:
d6d25f06-d091-4aa5-8826-6081a3a25525
    domainName: null
    id: Directory_FE2NN_24703906
    internalCorrelationId: null
    source: null
    targets: [ [+]
    ]
    tenantGeolocation: null
    tenantId: 225e05a1-5914-4688-a404-7030e60f3143
    tenantName: null
  }
```

Now confirm the account was actually disabled:

```
targets{}.modifiedProperties{}.newValue=["false"]
```

```
8/20/1      { [-]
8           activity: Disable account
2:47:12     activityDate: 2018-08-20T14:47:12.5151783Z
.515       activityDateInMillis: 1534776432515
PM          activityOperationType: Update
           activityResultDescription:
           activityResultStatus: Success

           activityType: User

           actor: { [-]

               @odata.type:
#Microsoft.ActiveDirectory.DataService.PublicApi.Model.Reporti
ng.AuditLog.ActorUserEntity

               ipAddress: <null>

               name: null

               objectId: 18af02a7-8541-4209-9dd5-600ab965d8a7

               puid: 1003BFFDA2E71FF9

               servicePrincipalName: null

               userPrincipalName: fyodor@froth.ly

           }

           actorType: User

           additionalDetails: [ [-]

       ]

       category: Core Directory

       componentOrSource: null

       correlationId: d6d25f06-d091-4aa5-8826-6081a3a25525

       domainName: null

       id: Directory_FE2NN_24703906

       internalCorrelationId: null

       source: null

       targets: [ [-]
```

```

{ [-]
  @odata.type:
  #Microsoft.ActiveDirectory.DataService.PublicApi.Model.Reporting.AuditLog.TargetResourceUserEntity
  additionalDetails: null
  ipAddress: null
  isPrivileged: null
  modifiedProperties: [ [+]
  ]
  name: null
  objectId: d85d399c-9cb6-480f-8789-88cbf56e3764
  puid: 10033FFFA361A98C
  targetResourceType: User
  userPrincipalName: bgist@froth.ly
  userSource: null
}
]
tenantGeolocation: null
tenantId: 225e05a1-5914-4688-a404-7030e60f3143
tenantName: null

```

It shows success but table it to make sure for easier viewing:

```
| table _time, activity, targets{}.userPrincipalName, actor.userPrincipalName
```

_time	activity	targets{}.userPrincipalName	actor.userPrincipalName
2018-08-20 14:47:12.515	Disable account	bgist@froth.ly	fyodor@froth.ly

bgist@froth.ly is successfully disabled at 2018-08-20 14:47:12.515 by fyodor@froth.ly

14.5 A phishing attack

14.5.1 Artifact from removed malicious content

14.5.1 Pinpointing an artifact associated with removed malicious content

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the filename (with extension) of the artifact left behind by the second phishing campaign sent to Frothy employees.

This will involve `stream:smtp` sourcetype and `base64_decode()` with the host as `matar`

Broad query:

```
index=client3 sourcetype="stream:smtp"
```

Narrow the search:

```
index=client3 sourcetype="stream:smtp" "financial model" "FY2019" "macros" "enable macros"
```

One event returned:

8/20/18

9:55:14.436 AM

{ [-]

attach_content_decoded_md5_hash: [[-]

395f9db4ccfeff4192fd50bdf13ae5e5

]

attach_content_md5_hash: [[-]

57976f601371ef529557fc513e3687b5

]

attach_disposition: [[-]

attachment

]

attach_filename: [[-]

Malware Alert Text.txt

]

attach_size: [[-]

272

]

attach_size_decoded: [[-]

197

]

attach_transfer_encoding: [[-]

base64

]

```
attach_type: [ [-]
  application/octet-stream
]
bytes: 5539
bytes_in: 5502
bytes_out: 37
content: [ [+]
]
content_body: [ [-]
  --_002_34d01f54b269459296334098605e3933DM6PR17MB2139namprd17pr_
```

Here is a financial model we can use for FY2019 planning. For the workshee=
t to operate properly, you will need to enable macros.
Thanks,Bruce =

```
.
]
content_transfer_encoding: quoted-printable
content_type: multipart/mixed;
  boundary="_002_34d01f54b269459296334098605e3933DM6PR17MB2139namprd17pr_"
date: Mon, 15 Sep 2018 01:21:12 +0000
dest_ip: 172.31.38.181
dest_mac: 06:6A:51:FA:0A:B0
dest_port: 25
endtime: 2018-08-20T09:55:14.436835Z
file_type: [ [+]
]
flow_id: 0c9748fa-4d85-4c7c-9042-07edafbe8d35
mime_type: multipart/mixed
mime_version: 1.0
msg_id:
<34d01f54-b269-4592-9633-4098605e3933@DM6PR17MB2139.namprd17.prod.outlook.com>
protocol_stack: ip:tcp:smtp
received_by_name: [ [+]
]
received_date: [ [+]
]
received_from_name: [ [+]
]
received_server_agent: [fe80::d97d:3d05:b533:b293%3]
received_with: [ [+]
```

```
]
receiver: [ [+]
]
receiver_alias: [ [+]
]
receiver_email: [ [+]
]
receiver_type: [ [+]
]
reply_time: 3284
request_time: 229
response_code: 250
response_time: 0
sender: Bruce Gist <bgist@froth.ly>
sender_alias: Bruce Gist
sender_email: bgist@froth.ly
server_response: 250 2.0.0 Ok: queued as 5204F1794E8
src_ip: 104.47.40.200
src_mac: 06:E3:CC:18:AA:33
src_port: 31712
subject: Draft Financial Plan for Brewery FY2019
time_taken: 3513
timestamp: 2018-08-20T09:55:14.433322Z
transport: tcp
}
```

Base64 imbedded into the event,

TWFsd2FyZSB3YXMGZGV0ZWN0ZWQgaW4gb25lIG9yIG1vcUgYXR0YWNobWVudHMgaW5jbHVkZWQg
d2l0aCB0aGlzIGVtYWlsIG1lc3NhZ2UuIA0KQWN0aW9uOiBBbGwgYXR0YWNobWVudHMgaGF2ZSBi
ZWVulHJlbW92ZWQudDQpGcm90aGx5LUJyZXdlcnktRmluYW5jaWFsLVBsYW5uaW5nLUZZMjAxOS1E
cmFmdC54bHNtCSBXOTdNLkVtcHN0YWdlIDQo=

Decoding in CyberChef:

Frothly-Brewery-Financial-Planning-FY2019-Draft.xlsm W97M.Empstage received at 8/20/18
9:55:14.436 AM

14.5.2 Executable embedded in malware

14.5.2 Unveiling the name of the executable embedded in the malware

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the name of the embedded executable (including file extension) found in the file I discovered in the previous task.

I'll be reviewing Sysmon Logs, looking for `EventCode=1` (process creation) and `EventCode=11` (file creation).

Query pivoting off of known file name:

```
index=client3 sourcetype=* "Frothly-Brewery-Financial-Planning-FY2019-Draft"
```

2 results

8/20/18

9:56:39.000 AM

08/19/2018 21:56:39 PM

LogName=Application

SourceName=Symantec AntiVirus

EventCode=51

EventType=2

Type=Error

ComputerName=BGIST-L.froth.ly

TaskCategory=The operation completed successfully.

OpCode=None

RecordNumber=7375

Keywords=Classic

Message=

Security Risk Found! W97M.Empstage in File:

C:\Users\BruceGist\AppData\Local\Packages\microsoft.windowscommunicationsapps_8wekyb3d8bbwe\LocalState\Files\S0\3\Frothly-Brewery-Financial-Planning-FY2019-Draft[66].xlsm by: Auto-Protect scan.

Action: Cleaned by Deletion. Action Description: The file was deleted successfully.

Collapse

host = BGIST-L index = client3 source = WinEventLog:Applications sourcetype = WinEventLog

8/20/18

9:55:52.000 AM

<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System><Provider

Name='Microsoft-Windows-Sysmon'


```

Guid='{5770385F-C22A-43E0-BF4C-06F5698FFBD9}'/><EventID>11</EventID><Version>2</Version><Level>4
</Level><Task>11</Task><Opcode>0</Opcode><Keywords>0x8000000000000000</Keywords><TimeCreated
SystemTime='2018-08-20T09:55:52.450060800Z'/'><EventRecordID>36035</EventRecordID><Correlation/'><
Execution ProcessID='3516'
ThreadID='5172'/'><Channel>Microsoft-Windows-Sysmon/Operational</Channel><Computer>BGIST-L.froth.ly</Computer><Security UserID='S-1-5-18'/'></System><EventData><Data Name='UtcTime'>2018-08-20
09:55:52.449</Data><Data
Name='ProcessGuid'>{EBF7A186-B00B-5B58-0000-0010CC954200}</Data><Data
Name='ProcessId'>10096</Data><Data Name='Image'>C:\Program
Files\WindowsApps\microsoft.windowscommunicationsapps_16005.10228.20127.0_x64__8wekyb3d8bbwe
\HxTsr.exe</Data><Data
Name='TargetFilename'>C:\Users\BruceGist\AppData\Local\Packages\microsoft.windowscommunications
apps_8wekyb3d8bbwe\LocalState\Files\S0\3\Frothly-Brewery-Financial-Planning-FY2019-Draft[66].xlsm</D
ata><Data Name='CreationUtcTime'>2018-08-20 09:55:52.449</Data></EventData></Event>

```

Now narrow for Sysmon file and process activity

```

index=client3 source="WinEventLog:Microsoft-Windows-Sysmon/Operational" host="fedora" (EventCode=1
OR EventCode=11)

```

Refine again

```

index=client3 source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
"Frothly-Brewery-Financial-Planning-FY2019-Draft"
(EventCode=1 OR EventCode=11)
| table _time, host, Image, TargetFilename, EventCode, ProcessId

```

_time	host	Image	TargetFilename	EventCode	ProcessId

20	B	C:\Program	C:\Users\BruceGist\AppData\Local\Pa	1	1
18	G	Files\WindowsApps\microsoft.	ckages\microsoft.windowscommunicat	1	0
-0	I	windowscommunicationsapps_	ionsapps_8wekyb3d8bbwe\LocalState\		0
8-	S	16005.10228.20127.0_x64__8w	Files\S0\3\Frothly-Brewery-Financial-Pl		9
20	T	ekyb3d8bbwe\HxTsr.exe	anning-FY2019-Draft[66].xlsm		6
09	-				
:5	L				
5:					
52					

Revealing the executable file as **HxTsr.exe**

14.5.3 Unique endpoints clicking the link

14.5.3 Calculating the number of unique endpoints that clicked the link

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out how many distinct IP addresses executed the malicious link file.

This involves looking in Office 365 logs using `o365:management:activity` or `ms:o365:management`.

Begin with broad query:

```
index=client3 sourcetype="ms:o365:management"
```

1073 events

Then isolate using `Operation=AnonymousLinkUsed`.

```
index=client3 sourcetype="ms:o365:management" Operation=AnonymousLinkUsed
```

11 results

Now count the unique client IPs that accessed the link:

```
| stats dc(ClientIP)
```

Results of **7**.

14.5.4 Port used to download pwntools

14.5.4 Confirming the port used to download malicious tools

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which port was used by the attacker to retrieve their attack toolkit.

I'll utilize `stream:*` logs for high-level network awareness as well as Sysmon EventCode 3 and apply the `| rare <field>`.

Broad query

```
index=client3 sourcetype=stream:* protocol_stack!=*udp*  
| rare dest_port
```

dest_port	count	percent
50504	1	0.001398
22790	2	0.002796
3333	2	0.002796
40552	2	0.002796
8088	2	0.002796
1025	3	0.004194

1080	3	0.004194
110	3	0.004194
111	3	0.004194
113	3	

Now look into the activity in these rare ports

```
index=client3 sourcetype=stream:* protocol_stack!=*udp* (dest_port=50504 OR dest_port=22790 OR
dest_port=40552 OR dest_port=8088 OR dest_port=1025 OR dest_port=1080 OR dest_port=3333) | table
_time dest_port host
```

I identify this host from earlier events

```
2018-08-20 10:48:57.322      3333  FYODOR-L
2018-08-20 10:47:16.891      3333  FYODOR-L
```

Pivot to Sysmon:

```
source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 DestinationPort=3333
```

Top values

Top values by time

Rare values

Events with this field

```
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

The attacker used port 3333

14.5.5 Remotely streamed files

14.5.5 Revealing remotely streamed files

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the names of the two files that were remotely streamed to **/tmp** during the intrusion.

This involves FIM and `osquery` since the server is Linux host= `hoth` and `wineventlog` sourcetype (typically from Windows hosts) can reveal how adversaries launch commands that target remote systems. In this task, identifying how PowerShell was used to stream content directly into the Linux server

Broad query:

```
index=client3 /tmp/* sourcetype!="ps" sourcetype!="lsf" NOT phpsessionclean  
388 events
```

Focus on suspicious Windows activity

```
index=client3 /tmp/* sourcetype="wineventlog"  
18 events
```

Now isolate unique command lines and reorder by time:

```
index=client3 /tmp/* sourcetype="wineventlog"  
| dedup Process_Command_Line  
| table _time, Process_Command_Line  
| reverse
```

7 results

2 echo results with large base 64 results

Since (`/tmp/colonel` OR `/tmp/*.sh`) happened immediately following the echos; pivot on those.

target_path: `/tmp/definitelydontinvestigatethisfile.sh` REDFLAG

Now confirm creation events

```
index=client3 sourcetype=osquery:results host=hoth /tmp/* (action=added OR action=modified)
```

colonel, [definitelydontinvestigatethisfile.sh](#) the 2 files load into `.tmp`

14.5.6 File containing malicious tools

14.5.6 Inferring which file contains the malicious tools

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which file likely contains the attacker's toolkit (based on earlier findings).

Again using rare dest_port and Interpret the stream:http sourcetype and the uri_path field
I know from previous pivots the port in question is 3333

Broad query:

```
index=client3 dest_port=3333
```

3 events

Narrow to HTTP

```
index=client3 dest_port=3333 sourcetype=stream:http
```

8/20/18

10:47:16.891 AM

{ [-]

bytes: 5542317

bytes_in: 177

bytes_out: 5542140

dest_ip: 45.77.53.176

dest_mac: 00:50:56:E3:C7:18

dest_port: 3333

endtime: 2018-08-20T10:47:16.891201Z

flow_id: e8947e90-2f4b-48eb-93e8-f0099d8f8188

http_comment: HTTP/1.1 200 OK

http_content_length: 5782482

http_content_type: image/png

http_method: GET

http_user_agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.17134.112

protocol_stack: ip:tcp:http

site: 45.77.53.176:3333

src_ip: 192.168.70.186

src_mac: 00:0C:29:55:51:1A

src_port: 64104

status: 200

time_taken: 11149728

timestamp: 2018-08-20T10:47:05.742156Z

transport: tcp

uri_path: /images/logos.png

}

logos.png contain the attackers toolkit.

14.5.7 Malware uploaded to DA's endpoint

14.5.7 Figuring out the first malicious executable uploaded to the DA's compromised endpoint

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the name of the first executable file uploaded to the domain admin's compromised machine.

I'll be looking into Sysmon Event ID 11,

First starting my query with the host involved. FYODOR-L
`index=client3 host=FYODOR-L`

Then narrow to event code 11

`index=client3 host=FYODOR-L EventCode=11 TargetFilename="*.exe" source="*sysmon*"`

The pipe into a clean table:

`| table _time, Image, TargetFilename, Hashes`

`| sort _time asc`

`| head 10`

_time	Image	TargetFilename	Hashes
2018-08-20 10:42:49	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\Temp\hdoor.exe	
2018-08-20 10:47:41	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\Temp\unzipped\Is of-master\iexplorer.exe	

2018-08-20 10:47:41	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\Temp\unzipped\ls of-master\powershell.exe
2018-08-20 11:34:24	C:\Windows\system32\cleanmgr.exe	C:\Users\FYODOR~1\AppData \Local\Temp\3F5D15FE-AD68 -4E1F-B3C4-90E199AF3640\D ismHost.exe

2018-08-20 10:42:49
C:\Windows\Temp\hdoor.exe

14.5.8 Geolocating attack country

14.5.8 Geolocating the country from which an attack is originating

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which country is the origin of a minor brute-force or password spray attempt targeting Frothly's web servers.

This will involve `linux_secure` sourcetype is used in Splunk to parse logs from the Linux `/var/log/secure` file. Filtering for `vendor_action="Invalid user"`. stats count by `src_ip` and using WHOIS or geo-IP lookups to find country of origin.

Broad query:

```
index=client3 sourcetype=linux_secure
```

Then filter for invalid user attempts

```
index=client3 sourcetype=linux_secure reason="Invalid user"
```

42 events

Dial it in clearing out noise

```
index=client3 sourcetype=linux_secure reason="Invalid user" tag=remote user!=admin user!=test
```

Now to see who made the most attempts

```
| stats count by src
```

```
| iplocation src
```

```
| sort -count
```

5.101.40.81	5.101.40.81 6	Moscow	Russia Moscow	55.79720	37.53780
-------------	---------------	--------	---------------	----------	----------

14.5.9 BBC rule created by attacker

14.5.9 Discovering details about a new BCC rule created by the adversary

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the value was assigned to the Name parameter in the adversary's BCC rule for exfiltrating Frothly's email.

Since this is for an email `ms:o365:management` sourcetype

Broad query:

```
index=client3 sourcetype=ms:o365:management
```

Dial in keywords:

```
("Frothly" OR "Name" OR "*Rule*" OR "*Blind*Carbon*Copy*")
```

Pipe into a table:

```
| table _time, UserId, Operation, Parameters, ResultStatus  
| sort _time asc
```

```
2018-08-20 11:21:40    fyodor@froth.ly New-TransportRule
```

Now reduce the scope to

```
index=client3 sourcetype=ms:o365:management  
Operation = New-TransportRule
```

Click on the `siemvid` field and `0e565fbe-511b-4fa6-9abb-14129861c04a`

8/20/18

{ [-]

11:21:40.000 AM

ClientIP: 199.66.91.253:40460

CreationTime: 2018-08-20T11:21:40

ExternalAccess: false

Id: f131587a-a125-4e87-4421-08d5f268e1ac

ObjectId:

Operation: New-TransportRule

OrganizationId:

225e05a1-5914-4688-a404-7030e60f3143

OrganizationName: frothly.onmicrosoft.com

OriginatingServer: CY4PR17MB1398 (15.20.0973.010)

Parameters: [[-]

{ [-]

Name: BlindCopyTo

Value: hyunki1984@naver.com

}

{ [-]

Name: Name

Value: SOX

}

{ [-]

Name: StopRuleProcessing

Value: False

}

{ [-]

Name: Mode

Value: Enforce

}

{ [-]

Name: Comments

```
      Value:
    }
  { [-]
    Name: RuleErrorAction
    Value: Ignore
  }
  { [-]
    Name: SenderAddressLocation
    Value: Header
  }
```

14.5.10 Post-pwn password creation

14.5.10 Determining the password of the user created post-compromise

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the password set for the new user created on the compromised endpoint.

Looking into WinEventLog:Security looking into the command line for new users

Broad query:

```
index=client3 "add" "user"
```

423 events

Reduce noise:

```
index=client3 "add" "user" NOT "Network Connected Devices Auto-Setup" NOT ec2-user  
sourcetype!="stream:mysql"
```

64 results

User created svcvnc 8/20/18 10:08:35.000 AM

Then query:

```
index=client3 source=WinEventLog:Microsoft-Windows-Sysmon/Operational host=FYODOR-L  
Sysmon/Operational svcvnc EventID=1
```

Review logs find password Password123!

14.6 Files, files, files

14.6.1 C2 server's URI

14.6.1 Retrieving the C2 server's URI

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the full URI path that the C2 server accessed.

This involves Sysmon Event ID 3 and PowerShell Script Block Logging (Event ID 4104)

First the broad query:

```
index=client3 source=WinEventLog:Microsoft-Windows-Sysmon/Operational EventID=3
```

5,072 events

Now sort by DestinationIp

```
index=client3 source=WinEventLog:Microsoft-Windows-Sysmon/Operational EventID=3  
| stats count by DestinationIp  
| sort -count
```

DestinationIp	count
45.77.53.176	4920
192.168.9.30	61
52.166.12.243	24
52.160.42.250	9

45.77.53.176 screams C2 host = FYODOR-L

now pivot on that host looking for events containing the C2IP

```
index=client3 source=WinEventLog:Microsoft-Windows-PowerShell/Operational EventCode=4104  
host=FYODOR-L
```

15 events

08/20/2018 11:32:14 AM

LogName=Microsoft-Windows-PowerShell/Operational
SourceName=Microsoft-Windows-PowerShell
EventCode=4104
EventType=3
Type=Warning
ComputerName=FYODOR-L.froth.ly
User=NOT_TRANSLATED
Sid=S-1-12-1-414122663-1107920193-174118301-2815976889
SidType=0
TaskCategory=Execute a Remote Command
OpCode=On create calls
RecordNumber=312
Keywords=None
Message=Creating Scriptblock text (1 of 1):
Inside the scriptblock is:
FromBase64String('aAB0AHQAcABzADoALwAvADQANQAuADcANwAuADUAMwAuADEANwA2ADoANAA0ADMA'));\$t='/news.php' decoded gives me <https://45.77.53.176:443>

Now to zero in on pivot FromBase64String:

index=client3 source=WinEventLog:Microsoft-Windows-PowerShell/Operational EventCode=4104
host=FYODOR-L FromBase64String

6 events

FromBase64String('aAB0AHQAcABzADoALwAvADQANQAuADcANwAuADUAMwAuADEANwA2ADoANAA0ADMA'));\$t='/admin/get.php' decoded gives me <https://45.77.53.176:443> when decoded using UTF-16LE

/admin/get.php being the full URL path for the C2 server accessed.

14.6.2 Hosts contacting C2 server

14.6.2 Identifying which hosts contacted the adversary's C2 server

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out which two short hostnames identify the Frothly endpoints that communicated with the adversary's C2 servers.

Again this involves WinEventLog:Microsoft-Windows-Sysmon/Operational source. Looking into any endpoint that may have accessed /admin/get.php

First a broad search:

index=client3 /admin/get.php

3 events

2018-08-20 10:07:07	FYODOR-L	WinEventLog:Microsoft-Windows-PowerShell/Operational	WinEventLog
2018-08-20 10:01:44	FYODOR-L	WinEventLog:Microsoft-Windows-PowerShell/Operational	WinEventLog
2018-08-20 10:01:00	ABUNGS-T-L	WinEventLog:Microsoft-Windows-PowerShell/Operational	WinEventLog

14.6.3 Contact info OSINT

14.6.3 Using OSINT to pivot off an indicator to find contact info

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the name of Al Bungstein's mobile service provider.

The sourcetype ms:o365:reporting:messagetrace captures detailed information about email traffic in Office 365, including originating IP addresses.

Broad query:

`index=client3 "Al Bungstein"`

Al's email `abungstein@froth.ly`

Possible IP as 104.47.32.48

Another broad search:

`index=client3 "abungstein@froth.ly"`

Review source types under fields

Values	Count	%
ms:o365:reporting:messagetrace	70	75.269%
ms:o365:management	12	12.903%
stream:smtp	11	11.828%

Pivot on `ms:o365:reporting:messagetrace`

`index=client3 "abungstein@froth.ly" sourcetype=ms:o365:reporting:messagetrace | stats count by FromIP | sort -count`

FromIP	count
174.215.1.81	40
107.77.212.175	9

91.207.175.56	7
104.207.83.63	3
107.77.213.173	3
107.77.213.96	2
12.196.122.127	2
104.238.59.42	1
12.196.122.120	1
45.56.149.8	1
91.207.175.249	1

Use ipinfo.io to look up 174.215.1.81

AS6167 - Verizon Business is the cell
carriers info

14.6.4 Contents of downloaded file

14.6.4 Revealing the contents of a downloaded file

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out what single word best captures how Mallory's research suggests beer should be enjoyed.
code42:security sourcetype logs file activity on endpoints, such as downloads, uploads, renames, and deletions.

First find out Mallory Kraeusen's email address, the same as Al's.

Guess is mkraeusen@froth.ly (correct)

Workstation name **mkraeus-L**

Pivot to:

index=client3 mkrae* AND (*.doc* OR *.ppt* OR *.xls*)

File name **Frothly_GABF_Deck-2018-MK.pptx**

code42:security sourcetype to trace file activity

index=client3 Frothly_GABF_Deck-2018-MK.pptx sourcetype="code42:security"

Found **ba_advertising_code_overview.pdf**

Downloaded at 8/20/18 10:31:14.000 AM

Google search of ba_advertising_code_overview.pdf and Mallory says beer should be enjoyed **responsibly**

14.6.5 Privesc via process creation logs

14.6.5 Inspecting process creation logs for details about the adversary's privesc method

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

I want to figure out the full content of the file that escalates tomcat8's privileges to root.

On Linux systems, attackers frequently exploit vulnerabilities or misconfigurations to elevate from users like tomcat8 to root.

First find the correct sourcetype for hoth and tomcat8

`index=client3 host=hoth tomcat8 | stats count by sourcetype`

sourcetype	count
Unix:ListeningPorts	10
Unix:Service	5
Unix:UserAccounts	5
Isof	11647 2
osquery:results	256
package	15
ps	1052
top	523

osquery:results 256

So using that pivot

```
index=client3 sourcetype=osquery:results
```

Looking into the fields III pivot columns.cmdline and table to correlate across a sequence of events:

```
index=client3 sourcetype="osquery:results" tomcat8 columns.cmdline=*
```

```
| table _time siemeverid host columns.path columns.pid columns.uid decorations.username columns.cmdline
```

```
| reverse
```

```
2018-08-20 11:48:38 a087fb04-1057-46a1-9d7b-19e6c0e8192c hoth /bin/rm 22999 0  
root "rm" "/usr/share/tomcat8/.bash_history" where privilege escalation occurs user changed from  
tomcat8 to root.
```

Reviewing the table:

```
2018-08-20T11:09:05.000+0000 attacker used obfuscated text to hide command * Ubuntu 16.04.4 kernel  
priv esc to escalates tomcat8's privileges to root
```

14.6.6 Command exec vulnerabilities

14.6.6 Using OSINT to establish the vulnerability used for command execution

Instructions for completing this task are located [here](#). Use this worksheet to keep notes as you complete it.

Upon completing OSINT of the bleidl vnik ubuntu 16.04 kernel exploit I found it is tied to **CVE-2017-16995**. This is a local privilege escalation vulnerability affecting Linux kernels prior to 4.14.8, and it was commonly used to obtain root access on systems like Ubuntu 16.04.4 running kernels below 4.4.0-116. The exploit, originally developed by @bleidl and later modified by vnik, targets a flaw in the eBPF verifier that allows arbitrary memory read and write access. This enables a standard user to overwrite credential structures in kernel memory and elevate privileges to root.

Key indicators include references to the eBPF subsystem ([linux/bpf.h](#)) and BPF syscalls within the exploit code. The attack works by triggering a sign-extension vulnerability in the verifier, gaining memory access, and modifying UID/GID values to 0.

To mitigate this vulnerability, systems should be updated to a patched kernel version (4.4.0-116 or newer) and restrict unprivileged BPF access using [kernel.unprivileged_bpf_disabled=1](#).